Implementasi Sistem SOAR Open-Source Berbasis n8n untuk Deteksi dan Respons Ancaman Malware dan Phishing dengan Mitigasi Aktif Human-in-the-Loop (Studi Kasus: CV Bali Handmade)

OLEH:
Ravi Arnan Irianto (2305551076)

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS UDAYANA
2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan kepada Tuhan Yang Maha Esa, atas segala berkat dan rahmat-Nya yang membantu penulis dalam menyelesaikan tugas akhir dengan judul **"Implementasi Sistem SOAR *Open-Source* Berbasis n8n untuk Deteksi dan Respons Ancaman Malware dan Phishing dengan Mitigasi Aktif *Human-in-the-Loop* (Studi Kasus: CV Bali Handmade)"** ini. Dalam penyelesaian tugas akhir ini, penulis juga mendapatkan banyak bantuan materil dan non materil dari berbagai pihak. Oleh karena itu, penulis ingin mengucapkan terima kasih kepada pihak-pihak sebagai berikut.

1. Bapak Ir. I Nyoman Piarsa, ST., MT., IPM. selaku Dosen Pembimbing I yang telah membimbing penulis selama penelitian dan penulisan tugas akhir ini, termasuk memberikan arahan mengenai perancangan mekanisme *Active Response human-in-the-loop*, efisiensi pemantauan armada *workstation*, serta arsitektur agen pemantau ringan.
2. Bapak A.A. Kt Agung Cahyawan Wiranatha, S.T., M.T. selaku Dosen Pembimbing II yang telah memberikan bimbingan, masukan, dan evaluasi berharga selama penyusunan tugas akhir ini.
3. Orang tua dan keluarga yang telah memberikan dukungan penuh kepada penulis selama menempuh perkuliahan.
4. Pihak CV Bali Handmade serta rekan-rekan penulis di lingkungan perkuliahan maupun di luar lingkungan perkuliahan yang telah memberikan bantuan dan dukungan selama penyusunan tugas akhir.

Penulis menyadari bahwa tugas akhir ini masih jauh dari kata sempurna. Oleh karena itu, penulis memohon maaf atas segala kekurangan dan berharap penelitian ini dapat memberikan manfaat bagi pembaca serta menjadi rujukan pengembangan di masa mendatang.

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
DAFTAR GAMBAR v
DAFTAR TABEL vi
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 4
1.3 Tujuan Penelitian 4
1.4 Manfaat Penelitian 5
1.5 Batasan Masalah 5
1.6 Sistematika Penulisan 6
BAB II TINJAUAN PUSTAKA 8
2.1 State of The Art 8
2.2 Konsep Keamanan Siber 13
2.3 Intrusion Detection and Prevention System (IDS/IPS) 13
2.4 Security Information and Event Management (SIEM) 14
2.5 Wazuh sebagai Platform SIEM dan HIDS 15
2.6 Security Orchestration, Automation, and Response (SOAR) 15
2.7 Deteksi Malware dan File Integrity Monitoring 16
2.8 Deteksi Phishing 17
2.9 Orkestrasi Workflow dengan n8n 18
2.10 Threat Intelligence dan Analisis LLM 18
2.11 Active Response Human-in-the-Loop dan Integrasi Telegram Bot 19
BAB III METODOLOGI PENELITIAN 21
3.1 Waktu dan Tempat Penelitian 21
3.2 Sumber Data 21
3.3 Instrumen Perancangan Sistem 22
3.3.1 Perangkat Keras (Hardware) 22
3.3.2 Perangkat Lunak (Software) 23
3.4 Tahapan Penelitian 23

3.5 Metode Pengembangan Sistem.....25
3.5.1 Analisis Kebutuhan (Requirement Analysis).....26
3.5.2 Desain Sistem (System Design).....28
3.5.3 Implementasi (Implementation).....28
3.5.4 Pengujian (Testing).....29
3.5.5 Pemeliharaan (Maintenance).....29
3.6 Desain Arsitektur Sistem.....30
3.7 Perancangan Pipeline Deteksi dan Klasifikasi Severity.....31
3.8 Desain Active Response Human-in-the-Loop dan Integrasi Telegram Bot.....35
3.9 Perancangan Modul Riwayat dan Monitoring Event.....37
DAFTAR PUSTAKA.....39

DAFTAR GAMBAR

Gambar 3.1 Tahapan Penelitian.....24
Gambar 3.2 Tahapan Model Waterfall.....26
Gambar 3.3 Desain Arsitektur Sistem SOAR.....30
Gambar 3.4 Alur Pipeline Deteksi End-to-End.....32
Gambar 3.5 Sequence Active Response Human-in-the-Loop.....35
Gambar 3.6 Desain Integrasi Telegram Bot.....37

DAFTAR TABEL

Tabel 2.1 Ringkasan Penelitian Terdahulu.....10
Tabel 2.2 Analisis Kesenjangan Penelitian (Gap Analysis).....11
Tabel 3.1 Spesifikasi Perangkat Keras.....22
Tabel 3.2 Daftar Perangkat Lunak.....23
Tabel 3.3 Matriks Tingkat Keyakinan dan Otonomi Respons.....33
Tabel 3.4 Pemetaan Cakupan Deteksi terhadap MITRE ATT&CK.....34
Tabel 3.5 Tabel Indikator Pengujian.....29

BAB I
PENDAHULUAN

Bab I memuat pendahuluan dari penelitian "Implementasi Sistem SOAR *Open-Source* Berbasis n8n untuk Deteksi dan Respons Ancaman Malware dan Phishing dengan Mitigasi Aktif *Human-in-the-Loop* (Studi Kasus: CV Bali Handmade)". Bagian pendahuluan membahas mengenai latar belakang, rumusan masalah, tujuan penelitian, manfaat penelitian, batasan masalah, dan sistematika penulisan dari penelitian ini. Berikut adalah pembahasannya.

1.1 Latar Belakang

Transformasi digital mendorong hampir setiap organisasi untuk menghadirkan layanannya secara daring, mulai dari katalog produk, transaksi penjualan, hingga komunikasi operasional. Untuk menekan biaya infrastruktur, banyak organisasi berskala kecil hingga menengah memanfaatkan perangkat keras yang sudah tersedia, termasuk laptop bekas, sebagai server operasional. Dalam penelitian ini, penulis memanfaatkan sebuah laptop ThinkPad lama sebagai perangkat server yang menjalankan seluruh komponen sistem keamanan. Konsekuensinya, layanan yang terhubung ke jaringan internet dapat diakses dari mana saja sepanjang waktu, namun sekaligus terekspos terhadap berbagai upaya pemindaian dan serangan siber.

Keterbukaan ini membawa risiko keamanan yang tidak dapat diabaikan. Selain sasaran bot pemindai otomatis pada jaringan publik, karyawan atau pengguna *workstation* dapat secara tidak sengaja mengunduh berkas berbahaya atau mengakses tautan penipuan yang mengatasnamakan pihak resmi. Dua vektor inilah yang menjadi ancaman paling dominan dialami organisasi skala kecil hingga menengah, yaitu ***malware*** dan ***phishing*** (NIST, 2024; Puteri et al., 2024). *Malware* yang menyusup melalui unduhan berkas, media penyimpanan eksternal (USB), maupun rantai eksekusi skrip dapat menjadi titik awal kompromi sistem dan penyanderaan data (*ransomware*), sedangkan *phishing* menjadi vektor rekayasa sosial paling umum untuk pencurian kredensial. Kedua jenis ancaman ini berkembang sangat cepat, sehingga deteksi pasif saja tidak lagi memadai; organisasi membutuhkan kemampuan untuk **merespons** insiden secara cepat, terukur, dan konsisten (Habibzadeh et al., 2026).

Persoalan menjadi lebih nyata ketika sebuah organisasi belum memiliki sistem pemantauan keamanan siber sama sekali. Kondisi tersebut dijumpai pada **CV Bali Handmade**, sebuah perusahaan pengrajin dan penjual produk *handmade* di Bali yang menjalankan layanan digital dan sejumlah *workstation* operasional. Berdasarkan hasil observasi dan konfirmasi dengan pihak perusahaan, infrastruktur digital pada organisasi tersebut belum memiliki mekanisme pencatatan maupun pemantauan serangan siber yang terstruktur. Apabila terjadi unduhan berkas berbahaya atau akses ke tautan *phishing* pada perangkat karyawan, administrator tidak memperoleh peringatan dini dan tidak memiliki jejak audit (*log*) yang dapat ditelusuri. Kondisi ini membuat sistem berada dalam keadaan rentan tanpa disadari. Oleh karena itu, penelitian ini membangun sistem keamanan terpusat pada laptop ThinkPad lama milik peneliti sebagai server, yang dirancang khusus untuk menjawab kebutuhan pemantauan multi-*endpoint* pada CV Bali Handmade.

Dalam menghadapi ancaman tersebut, pendekatan *Security Operations Center* (SOC) konvensional yang hanya mengandalkan *Security Information and Event Management* (SIEM) seperti Wazuh umumnya berhenti pada tahap pengumpulan *log* dan pembuatan *alert* (Alves et al., 2026). Proses verifikasi reputasi indikator ancaman (*enrichment*), penentuan tingkat keparahan, hingga eksekusi tindakan penanggulangan masih harus dilakukan secara manual oleh administrator. Pendekatan manual ini lambat, tidak konsisten, dan memicu kelelahan analis (*alert fatigue*) akibat banyaknya peringatan positif palsu (*false positive*) dari perubahan berkas normal (Chhetri et al., 2024; Tariq et al., 2025). Di sisi lain, platform *Security Orchestration, Automation, and Response* (SOAR) komersial seperti Splunk SOAR atau Cortex XSOAR memiliki biaya lisensi yang tinggi sehingga tidak terjangkau bagi usaha kecil dan menengah (Nadig & Vignesh, 2026). Sebaliknya, penerapan otomatisasi respons yang sepenuhnya buta (*fully automated response*) tanpa keterlibatan manusia juga berisiko tinggi menimbulkan gangguan operasional apabila sistem secara keliru mengisolasi berkas konfigurasi sistem operasi atau memblokir layanan yang sah (Mohsin et al., 2025).

Untuk menjembatani kesenjangan antara kecepatan otomatisasi dan keamanan kendali manusia, penelitian ini mengusulkan implementasi sistem **SOAR *open-source* berbasis n8n** dengan mekanisme **respons berjenjang berbasis tingkat keyakinan (*confidence-based tiered response*)** dan **mitigasi aktif *human-in-the-loop* (HITL)**. Sistem memadukan **Wazuh** sebagai

SIEM dan *Host-based Intrusion Detection System* (HIDS) beserta alternatif **agen pemantau ringan berbasis Rust (soar-agent)** untuk perangkat dengan sumber daya terbatas, **n8n** sebagai mesin orkestrasi *playbook*, verifikasi *threat intelligence* multi-sumber (VirusTotal, MalwareBazaar, AlienVault OTX, dan umpan OSINT MISP untuk reputasi berkas, serta Google Safe Browsing, URLScan.io, dan URLhaus untuk reputasi URL), **Large Language Model (LLM)** sebagai modul analisis kontekstual berbahasa Indonesia, serta **Telegram Bot** dan **Dashboard Pemantauan Armada (*Fleet Monitor*)** sebagai antarmuka pemantauan dan pengambilan keputusan dua arah. Seluruh komunikasi antar-jaringan diamankan menggunakan jaringan privat *Tailscale* dan *Cloudflare Access*. Sesuai prinsip *trusted autonomy* (Mohsin et al., 2025; Singh et al., 2025), ancaman dengan bukti konsensus tinggi ditangani secara cepat, sedangkan ancaman dengan indikasi sedang atau belum terverifikasi diarahkan ke jalur *human-in-the-loop* melalui tombol keputusan interaktif pada Telegram yang dilengkapi batas waktu (*Service Level Agreement* / SLA) eskalasi otomatis.

Sejumlah penelitian terdahulu telah mengkaji pemanfaatan Wazuh dan integrasi SOAR maupun kanal notifikasi. Farrel dkk. (2024) dan Ariyanto dkk. (2025) mengintegrasikan Wazuh dengan fitur *Active Response* otomatis dan notifikasi Telegram untuk menangani serangan *brute force* serta DDoS. Saputra dkk. (2024) dan Antara & Rachmawati (2024) memanfaatkan Wazuh untuk deteksi intrusi dan pengiriman notifikasi satu arah ke Telegram maupun Discord. Pada ranah SOAR, Thakker dkk. (2025) dan Nadig & Vignesh (2026) mengintegrasikan Wazuh dengan platform Shuffle untuk respons otomatis terhadap serangan lapisan aplikasi. Meskipun demikian, penelitian-penelitian tersebut masih menerapkan otomatisasi biner (notifikasi satu arah saja atau eksekusi otomatis penuh tanpa verifikasi multi-sumber), belum memetakan tingkat keyakinan ancaman terhadap tingkat otonomi tindakan, serta belum menggabungkan penanganan *malware* dan *phishing* dengan analisis LLM dan kendali *human-in-the-loop* dua arah pada skenario organisasi nyata.

Berdasarkan celah penelitian tersebut, penelitian ini merancang, mengimplementasikan, dan mengevaluasi sistem SOAR *open-source* berbasis n8n yang utuh pada studi kasus CV Bali Handmade. Sistem divalidasi secara menyeluruh melalui simulasi serangan terkendali untuk mengukur kinerja kecepatan deteksi, waktu tanggap (*Mean Time to Respond*), reduksi positif

palsu, konsistensi analisis LLM, serta keandalan eksekusi mitigasi aktif sebelum diterapkan pada lingkungan operasional perusahaan.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut.

1. Bagaimana merancang dan membangun arsitektur sistem SOAR *open-source* berbasis n8n yang mengintegrasikan Wazuh dan agen pemantau ringan sebagai sensor *endpoint*, n8n sebagai mesin orkestrasi, *threat intelligence* multi-sumber (VirusTotal, MalwareBazaar, AlienVault OTX, MISP, Google Safe Browsing, URLScan.io, dan URLhaus), modul analisis LLM, serta Telegram Bot dan *dashboard* pemantauan pada server berbasis laptop ThinkPad lama?
2. Bagaimana mengorkestrasi *pipeline* deteksi ancaman *malware* dan *phishing*, melakukan verifikasi reputasi multi-sumber dengan mekanisme *cache* bertingkat, serta memetakan tingkat keyakinan ancaman terhadap tingkat otonomi respons secara *real-time*?
3. Bagaimana merancang mekanisme *Active Response human-in-the-loop* dua arah melalui Telegram beserta pencatatan *registry* keputusan dan eskalasi batas waktu (SLA) agar tindakan mitigasi kritis seperti karantina berkas dan pemblokiran domain tetap terkendali oleh analis manusia?
4. Bagaimana kinerja sistem yang dibangun diukur secara kuantitatif menggunakan metrik *Mean Time to Detect* (MTTD), *Mean Time to Respond* (MTTR), tingkat reduksi *false positive*, tingkat *false negative*, konsistensi keluaran LLM, serta keberhasilan eksekusi respons pada skenario pengujian?

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah tersebut, tujuan yang hendak dicapai dalam penelitian ini adalah sebagai berikut.

1. Merancang dan membangun sistem SOAR *open-source* berbasis n8n yang memadukan Wazuh, agen pemantau ringan berbasis Rust, *threat intelligence* multi-sumber, modul analisis LLM, *dashboard* pemantauan, dan Telegram Bot pada server berbasis laptop ThinkPad lama.

2. Menerapkan *pipeline* deteksi *malware* dan *phishing* dengan pengayaan *threat intelligence* multi-sumber dan matriks pemetaan tingkat keyakinan terhadap tingkat otonomi respons secara *real-time*.
3. Mengimplementasikan mekanisme *Active Response human-in-the-loop* dua arah melalui Telegram yang dilengkapi *registry* audit keputusan analis, fitur pemulihan karantina, serta eskalasi SLA otomatis.
4. Mengevaluasi kinerja sistem secara kuantitatif berdasarkan metrik MTTD, MTTR, reduksi *false positive*, *false negative*, konsistensi analisis LLM, serta ketahanan terhadap beban *alert* serentak.

1.4 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat teoretis maupun praktis sebagai berikut.

1. **Manfaat Teoretis:** memberikan kontribusi keilmuan pada bidang keamanan jaringan dan operasional SOC mengenai perancangan SOAR *open-source* berbasis *trusted autonomy* yang memadukan koroborasi *threat intelligence* multi-sumber, analisis kontekstual LLM berbatas aman (*guardrail*), serta matriks *confidence-to-autonomy* untuk menyeimbangkan kecepatan otomatisasi dan kendali manusia (*human-in-the-loop*).
2. **Manfaat Praktis:** menyediakan solusi pemantauan dan respons insiden siber berbiaya rendah tanpa biaya lisensi komersial yang dapat diterapkan pada CV Bali Handmade, sehingga administrator memperoleh peringatan dini yang bebas dari kebisingan (*noise*), memiliki riwayat insiden yang dapat ditelusuri melalui *dashboard*, serta dapat mengeksekusi maupun memulihkan tindakan mitigasi langsung dari perangkat bergerak melalui Telegram.

1.5 Batasan Masalah

Agar pembahasan dan evaluasi penelitian tetap terarah, ruang lingkup penelitian ini dibatasi pada ketentuan berikut.

1. Komponen utama sistem yang digunakan meliputi Wazuh (versi 4.14.8), agen pemantau ringan berbasis Rust (`soar-agent`), n8n (versi 2.42.5), sumber *threat intelligence* (VirusTotal, MalwareBazaar, AlienVault OTX, umpan OSINT MISP, Google Safe

Browsing, URLScan.io, dan URLhaus), API LLM, *dashboard* pemantauan (`fleet-monitor`), serta Telegram Bot.

2. Penelitian berfokus pada implementasi dan evaluasi orkestrasi deteksi serta respons berbasis *stack open-source*, bukan bertujuan menggantikan perangkat *network intrusion detection system* (NIDS) berbasis *signature* jaringan seperti Snort atau Suricata.
3. Studi kasus penelitian berfokus pada kebutuhan pemantauan keamanan *endpoint* di CV Bali Handmade, dengan server pusat dijalankan pada laptop ThinkPad lama milik peneliti.
4. Vektor ancaman yang diujikan dibatasi pada dua kelas ancaman utama, yaitu *malware* (melalui *File Integrity Monitoring*, deteksi berkas umpan *canary ransomware*, serta aturan rantai proses *Living-off-the-Land Binaries* / LOLBin) dan *phishing* (melalui analisis *log* akses tautan/URL, berkas pintasan berbahaya, dan umpan URL proaktif); serangan penolakan layanan jaringan berskala besar (DDoS volumetrik) maupun eksploitasi *kernel memory-only* berada di luar cakupan.
5. Tindakan *Active Response* yang diimplementasikan meliputi isolasi/karantina berkas (`quarantine-file`) beserta mekanisme pemulihannya (*restore*) untuk ancaman *malware*, serta pemblokiran domain melalui *sinkhole* (`/etc/hosts block-domain`) untuk ancaman *phishing*, dengan perlindungan yang melarang karantina otomatis terhadap berkas konfigurasi sistem operasi maupun proses sistem bawaan.
6. Pengayaan *threat intelligence* eksternal memanfaatkan kuota API *tier* komunitas/gratis yang dimitigasi dengan mekanisme *cache* sisi server ber-TTL diferensial dan *fallback* lintas sumber, sedangkan analisis kontekstual dijalankan melalui API penyedia LLM dengan konteks *playbook* lokal.
7. Pengujian serangan dan pengukuran metrik dilakukan pada lingkungan simulasi terkendali untuk menjamin keselamatan operasional sebelum sistem diterapkan pada perangkat produksi perusahaan.

1.6 Sistematika Penulisan

Sistematika penulisan laporan tugas akhir ini disusun ke dalam lima bab utama sebagai berikut.

1. **BAB I PENDAHULUAN:** Memuat latar belakang permasalahan, rumusan masalah, tujuan penelitian, manfaat penelitian, batasan masalah, dan sistematika penulisan.
2. **BAB II TINJAUAN PUSTAKA:** Memuat kajian penelitian terdahulu (*state of the art*) beserta tabel analisis kesenjangan (*gap analysis*), dan landasan teori yang meliputi konsep keamanan siber, IDS/IPS, SIEM, Wazuh dan agen pemantau ringan, SOAR, deteksi *malware* dan kerangka MITRE ATT&CK, deteksi *phishing*, orkestrasi n8n, *threat intelligence* multi-sumber dan analisis LLM, serta *Active Response human-in-the-loop*.
3. **BAB III METODOLOGI PENELITIAN:** Memuat waktu dan tempat penelitian, sumber data, instrumen perangkat keras dan perangkat lunak, tahapan penelitian, metode pengembangan *Waterfall*, desain arsitektur berlapis, perancangan *pipeline* dan matriks otonomi, desain *Active Response human-in-the-loop* dan eskalasi SLA, serta perancangan modul riwayat dan *monitoring* armada.
4. **BAB IV HASIL DAN PEMBAHASAN:** Memuat hasil implementasi sistem secara menyeluruh serta analisis hasil pengujian kuantitatif berdasarkan metrik evaluasi yang telah ditetapkan.
5. **BAB V PENUTUP:** Memuat kesimpulan akhir yang menjawab rumusan masalah serta saran pengembangan sistem untuk penelitian selanjutnya.

BAB II
TINJAUAN PUSTAKA

2.1 State of The Art

Kajian pustaka dilakukan terhadap sejumlah penelitian jurnal ilmiah terbaru (rentang tahun 2023 hingga 2026) yang membahas implementasi SIEM Wazuh, orkestrasi SOAR, integrasi *threat intelligence*, pemanfaatan *Large Language Model* (LLM) pada operasional keamanan, serta mekanisme respons berbasis Telegram Bot. Ulasan berikut merangkum kontribusi sekaligus keterbatasan dari masing-masing penelitian terdahulu sebagai dasar penentuan kebaruan (*novelty*) penelitian ini.

Penelitian pertama dilakukan oleh Prasetyo dkk. (2023) mengenai evaluasi kinerja *Host-based Intrusion Detection System* (HIDS) Wazuh dalam mendeteksi serangan *Brute Force* (menggunakan Medusa dan Hydra) serta *Denial of Service* (menggunakan Hping3 dan Metasploit). Hasil pengujian menunjukkan bahwa Wazuh mampu mendeteksi serangan *Brute Force* secara *real-time* dan memetakannya ke dalam skema MITRE ATT&CK, namun belum dilengkapi mekanisme pengayaan reputasi eksternal maupun orkestrasi respons aktif untuk mengisolasi ancaman pada *endpoint* (Prasetyo et al., 2023).

Penelitian kedua dilakukan oleh Farrel dkk. (2024) yang menerapkan Wazuh sebagai SIEM dengan fitur *Active Response* dan integrasi Telegram API untuk memitigasi serangan *brute force* pada sistem informasi GT-I2TI Universitas Trisakti menggunakan metodologi *Security Lifecycle Model* (*Identify, Assess, Protect, Monitor*). Sistem berhasil mendeteksi 100% serangan *brute force* dengan rata-rata waktu deteksi 80,51 detik dan waktu reaksi *Active Response* pemblokiran IP otomatis sebesar 0,51 detik (Farrel et al., 2024). Meskipun respons berlangsung cepat, fitur *Active Response* dijalankan secara otomatis penuh tanpa verifikasi analis dan notifikasi Telegram hanya bersifat satu arah, sehingga berisiko memblokir pengguna sah apabila terjadi *false positive*.

Penelitian ketiga dilakukan oleh Saputra dkk. (2024) mengenai implementasi Wazuh SIEM yang diintegrasikan dengan Telegram Bot untuk deteksi, analisis, dan visualisasi *log* keamanan secara *real-time* pada Pesantren Teknologi Informasi dan Komunikasi (PeTIK) Jombang terhadap serangan *Brute Force*, *SYN Flood*, dan *SQL Injection* (Saputra et al., 2024).

Sejalan dengan itu, penelitian keempat oleh Antara & Rachmawati (2024) menguji efektivitas Wazuh sebagai IDS untuk mendeteksi serangan *Local File Inclusion* (LFI), *XML External Entity* (XXE), dan DoS pada server lingkungan rumah sakit dengan pengiriman notifikasi *real-time* melalui Discord (Antara & Rachmawati, 2024). Kedua penelitian tersebut membuktikan keandalan Wazuh dalam mendeteksi insiden dan mengirim peringatan ke kanal pesan instan, tetapi keduanya berhenti pada notifikasi satu arah tanpa kemampuan eksekusi mitigasi balik (*two-way response*) dari kanal pesan yang sama.

Penelitian kelima dilakukan oleh Ariyanto dkk. (2025) yang mengembangkan sistem deteksi dan tanggapan ancaman siber otomatis berbasis Wazuh SIEM, Fail2Ban, klasifikasi *machine learning* (*Random Forest*), serta Telegram API dengan metodologi PPDIOO (*Prepare, Plan, Design, Implement, Operate, Optimize*). Sistem menghasilkan akurasi deteksi 98,4% terhadap pola serangan *Brute Force* dan DDoS dengan waktu tanggap di bawah 10 detik (Ariyanto et al., 2025). Penelitian keenam oleh Artika dkk. (2025) mengembangkan sistem keamanan web *SecurityShield* menggunakan algoritma *Isolation Forest* yang dilengkapi autentikasi OTP Telegram dan *dashboard* WebSocket *real-time* (Artika et al., 2025). Kedua penelitian ini menunjukkan kematangan integrasi analitik dengan Telegram, namun belum mengadopsi mesin orkestrasi *playbook* SOAR untuk menangani *malware* berbasis berkas maupun tautan *phishing* dengan koroborasi *threat intelligence* multi-sumber.

Pada ranah integrasi SIEM dan SOAR *open-source*, penelitian ketujuh oleh Thakker dkk. (2025) mengintegrasikan Wazuh dengan platform SOAR Shuffle untuk pertahanan otomatis terhadap serangan lapisan aplikasi pada sistem Windows, yang berhasil menurunkan waktu penanganan insiden secara signifikan dibandingkan triase manual (Thakker et al., 2025). Selanjutnya, penelitian kedelapan oleh Nadig & Vignesh (2026) memperluas kapabilitas Wazuh SIEM melalui integrasi SOAR untuk mengotomasi pengayaan dan eksekusi respons ancaman tanpa intervensi manual (Nadig & Vignesh, 2026), sedangkan penelitian kesembilan oleh Alves dkk. (2026) merancang arsitektur deteksi insiden berbasis Wazuh terdistribusi yang berfokus pada skalabilitas pengumpulan dan korelasi *alert* (Alves et al., 2026). Kendati demikian, implementasi SOAR pada penelitian Thakker dkk. (2025) dan Nadig & Vignesh (2026) masih bertumpu pada otomatisasi penuh tanpa kendali *human-in-the-loop* dua arah dan belum menyertakan analisis kontekstual berbasis LLM.

Di sisi kolaborasi manusia dan kecerdasan buatan pada SOC, penelitian kesepuluh oleh Mohsin dkk. (2025) merumuskan kerangka kerja *Trusted Autonomy* dalam kolaborasi manusia dan AI di SOC. Studi tersebut menegaskan bahwa otomatisasi keamanan tidak seharusnya bersifat biner (sepenuhnya manual atau sepenuhnya otomatis), melainkan harus memetakan tingkat otonomi sistem terhadap tingkat keyakinan (*confidence*) bukti ancaman serta risiko dampak operasional (Mohsin et al., 2025). Temuan tersebut diperkuat oleh studi empiris Singh dkk. (2025) terhadap 3.090 kueri dari 45 analis SOC serta survei komprehensif Habibzadeh dkk. (2026) dan Srinivas dkk. (2025), yang membuktikan bahwa LLM paling efektif dan aman ketika diposisikan sebagai asisten kognitif (*advisory*) untuk merangkum bukti dan menjelaskan alasan keputusan (*explainability*), bukan sebagai eksekutor tindakan destruktif secara otonom karena adanya risiko halusinasi model dan *prompt injection* (Hammar et al., 2025; Singh et al., 2025).

Untuk memberikan gambaran komparatif yang terstruktur, ringkasan fokus, metode, dan mekanisme respons dari penelitian-penelitian terdahulu disajikan pada Tabel 2.1.

Tabel 2.1 Ringkasan Penelitian Terdahulu

No	Peneliti dan Tahun	Fokus Utama	Metode dan Komponen	Mekanisme Respons dan Alert
1	Prasetyo dkk. (2023)	Evaluasi kinerja HIDS Wazuh terhadap serangan *Brute Force* dan DoS	Wazuh HIDS, pemetaan aturan MITRE ATT&CK	Deteksi di *dashboard* Wazuh; tanpa notifikasi eksternal maupun *Active Response*
2	Farrel dkk. (2024)	Mitigasi serangan *brute force* pada sistem informasi akademik	Wazuh SIEM, *Active Response* bawaan, Telegram Bot API	Pemblokiran IP otomatis penuh; notifikasi Telegram satu arah
3	Saputra dkk. (2024)	Deteksi dan visualisasi *log* keamanan secara *real-time*	Wazuh SIEM, integrasi kustom Telegram Bot	Notifikasi Telegram satu arah dan visualisasi *dashboard*; tanpa mitigasi aktif
4	Antara & Rachmawati (2024)	Deteksi intrusi LFI, XXE, dan DoS pada jaringan rumah sakit	Wazuh IDS pada kontainer Docker, webhook Discord	Notifikasi Discord satu arah; tanpa orkestrasi SOAR maupun *Active Response*
5	Thakker dkk. (2025)	Pertahanan otomatis serangan *application-layer* pada Windows	Wazuh SIEM dan platform SOAR Shuffle	Eksekusi *playbook* otomatis penuh berbasis aturan; tanpa persetujuan analis
6	Ariyanto dkk. (2025)	Deteksi dan respons ancaman *Brute Force* serta DDoS otomatis	Wazuh SIEM, Fail2Ban, *Random Forest*, Telegram API	Pemblokiran IP dinamis otomatis; notifikasi Telegram satu arah
7	Artika dkk. (2025)	Deteksi anomali *log* aktivitas aplikasi web	*Isolation Forest*, WebSocket, autentikasi	Peringatan *real-time* dan OTP Telegram; khusus anomali

No	Peneliti dan Tahun	Fokus Utama	Metode dan Komponen	Mekanisme Respons dan Alert
		secara *real-time*	OTP Telegram	aplikasi web
8	Mohsin dkk. (2025) & Singh dkk. (2025)	Kerangka *Trusted Autonomy* dan kolaborasi manusia-LLM di SOC	Pemodelan tingkat otonomi berbasis *confidence* dan evaluasi empiris LLM di SOC	Kerangka konseptual dan studi empiris; LLM sebagai *advisory* dengan kendali HITL
9	Nadig & Vignesh (2026)	Penguatan kapabilitas SIEM melalui integrasi SOAR	Wazuh SIEM dan mesin orkestrasi SOAR Shuffle	Respons otomatis penuh untuk mereduksi triase manual; tanpa *human-in-the-loop*
10	Alves dkk. (2026)	Perancangan deteksi insiden terdistribusi berbasis Wazuh	Klaster Wazuh Manager, Indexer, dan multi-*agent*	Berfokus pada arsitektur deteksi dan ketersediaan tinggi; minim orkestrasi respons

Berdasarkan pemetaan pada Tabel 2.1, dilakukan analisis kesenjangan (*gap analysis*) untuk mengidentifikasi keterbatasan spesifik pada setiap penelitian terdahulu serta merumuskan kontribusi yang dihadirkan oleh penelitian ini, sebagaimana dijabarkan pada Tabel 2.2.

Tabel 2.2 Analisis Kesenjangan Penelitian (*Gap Analysis*)

Sumber Penelitian	Kesenjangan (*Gap*) Penelitian Terdahulu	Kontribusi pada Penelitian Ini
Prasetyo dkk. (2023); Saputra dkk. (2024); Antara & Rachmawati (2024)	Hanya berhenti pada tahap deteksi dan notifikasi satu arah; administrator tetap harus membuka *dashboard* atau masuk ke server secara manual untuk menindaklanjuti insiden.	Menghadirkan orkestrasi SOAR *end-to-end* berbasis n8n dengan kanal keputusan dua arah melalui tombol interaktif Telegram yang langsung memicu *Active Response* pada *endpoint*.
Farrel dkk. (2024); Ariyanto dkk. (2025)	Menerapkan *Active Response* otomatis penuh (pemblokiran IP) hanya berdasarkan aturan SIEM tanpa verifikasi reputasi eksternal, sehingga rentan mengeksekusi tindakan secara buta terhadap *false positive* dan hanya mencakup serangan jaringan/autentikasi.	Menerapkan verifikasi *threat intelligence* multi-sumber sebelum merespons, melindungi berkas konfigurasi sistem dari karantina salah sasaran, serta memperluas cakupan ke dua kelas ancaman *endpoint* (*malware* dan *phishing*).
Thakker dkk. (2025); Nadig & Vignesh (2026)	Menggunakan mesin SOAR Shuffle dengan pola otomatisasi penuh tanpa kendali *human-in-the-loop*, belum memiliki matriks pemetaan keyakinan ke otonomi, serta belum mengintegrasikan analisis kontekstual LLM.	Mengimplementasikan n8n sebagai mesin SOAR dengan matriks *confidence-to-autonomy* (otonom untuk bukti kuat, HITL dengan SLA 15 menit untuk bukti sedang) dan ringkasan LLM berbahasa Indonesia berbasis *playbook* lokal.
Alves dkk. (2026); Artika dkk. (2025)	Berfokus pada lapisan deteksi SIEM atau anomali web saja tanpa orkestrasi *playbook* lintas *threat intelligence* dan tanpa perbandingan beban agen pada perangkat bersumber daya rendah.	Menambahkan lapisan orkestrasi SOAR di atas Wazuh sekaligus menyediakan opsi agen pemantau ringan berbasis Rust (`soar-agent`, ~5 MB) untuk *workstation* dengan keterbatasan memori.
Mohsin dkk. (2025); Singh dkk. (2025); Hammar dkk. (2025)	Menyajikan kerangka konseptual *trusted autonomy* dan evaluasi LLM di SOC, namun belum diwujudkan menjadi implementasi teknis *open-source* terpadu yang diuji secara empiris pada studi kasus UKM.	Mewujudkan konsep *trusted autonomy* dan *AI-advisory* ke dalam sistem nyata yang dilengkapi *registry* audit keputusan, *guardrail* anti-halusinasi LLM, serta evaluasi kuantitatif pada studi kasus CV Bali Handmade.

Berdasarkan analisis kesenjangan pada Tabel 2.2, kebaruan (*novelty*) penelitian ini bersifat integratif dan metodologis, yang mencakup enam aspek utama sebagai berikut.

1. **Verifikasi Ancaman Multi-Sumber dengan Cache Diferensial:** Menggabungkan empat sumber reputasi berkas (VirusTotal, MalwareBazaar, AlienVault OTX, dan umpan OSINT MISP) serta tiga sumber reputasi URL (Google Safe Browsing, URLScan.io, dan URLhaus) yang dilengkapi *cache* sisi server ber-TTL diferensial untuk menekan *alert fatigue* dan menutup celah *false negative* dari ketergantungan pada satu sumber tunggal.
2. **Matriks *Confidence-to-Autonomy* (Respons Berjenjang):** Memformalkan aturan tunggal yang memetakan kekuatan bukti ancaman ke dalam tingkat otonomi tindakan (`auto-critical`, `auto-high`, `hitl-review`, `hitl-chain`, `manual-config`, dan `observe`), sehingga perubahan berkas biasa tidak lagi divonis secara keliru sebagai *malware*.
3. **Mitigasi Aktif *Human-in-the-Loop* Dua Arah Ber-SLA dan Ber-Audit:** Menyediakan tombol keputusan langsung pada notifikasi Telegram yang terhubung ke *registry* keputusan persisten (`alert-log`), dilengkapi mekanisme pemulihan berkas karantina (*restore*) serta alur kerja eskalasi otomatis apabila analis tidak merespons dalam batas waktu 15 menit.
4. **Penanganan Dua Kelas Ancaman dan Rantai Proses Eksekusi:** Menangani ancaman *malware* (termasuk deteksi ekstensi ganda, berkas umpan *canary ransomware*, dan rantai proses LOLBin melalui Sysmon) dengan aksi `quarantine-file`, serta ancaman *phishing* (reaktif dan proaktif) dengan aksi *sinkhole* domain (`block-domain`).
5. **Analisis Kontekstual LLM Berbasis Bukti dan RAG Playbook Lokal:** Memanfaatkan model bahasa melalui API dengan penyuntikan konteks *playbook* penanganan insiden lokal (*Retrieval-Augmented Generation* ringan) dan *guardrail* ketat agar AI berperan murni sebagai penjelas keputusan (*explainability*) dalam Bahasa Indonesia tanpa pernah mengeksekusi *Active Response* secara mandiri.
6. **Arsitektur Hybrid Multi-Agent dan Pemantauan Armada Terpusat:** Mendukung pemantauan lintas sistem operasi (Linux dan Windows) baik menggunakan Wazuh Agent maupun agen ringan mandiri berbasis Rust (`soar-agent`), yang dipantau secara terpusat melalui *dashboard* web dan antarmuka terminal (TUI).

2.2 Konsep Keamanan Siber

Keamanan siber (*cybersecurity*) merupakan kumpulan teknologi, kebijakan, proses, dan praktik yang dirancang untuk melindungi jaringan, perangkat komputasi, program, serta data dari serangan, kerusakan, maupun akses tanpa izin. Menurut kerangka kerja *NIST Cybersecurity Framework (CSF) 2.0*, pengelolaan risiko keamanan siber yang komprehensif bertumpu pada enam fungsi inti yang saling berkesinambungan, yaitu *Govern, Identify, Protect, Detect, Respond,* dan *Recover* (NIST, 2024; Puteri et al., 2024). Prinsip dasar perlindungan informasi berlandaskan pada *CIA Triad*, yang meliputi tiga pilar utama sebagai berikut.

1. ***Confidentiality*** **(Kerahasiaan):** menjamin bahwa informasi sensitif dan kredensial hanya dapat diakses oleh pihak yang memiliki otorisasi sah, yang sering kali menjadi target utama dari serangan rekayasa sosial *phishing*.
2. ***Integrity*** **(Integritas):** menjaga keutuhan dan keakuratan sistem serta berkas dari modifikasi tanpa izin, penyisipan *malware*, maupun manipulasi konfigurasi sistem operasi.
3. ***Availability*** **(Ketersediaan):** memastikan sistem, jaringan, dan data tetap dapat digunakan oleh pengguna yang berhak ketika dibutuhkan, serta terlindungi dari gangguan layanan maupun penyanderaan berkas oleh *ransomware*.

Dalam konteks penelitian ini, sistem SOAR yang dirancang berperan langsung dalam menjembatani fungsi *Detect* (melalui pemantauan integritas berkas dan analisis *log* akses tautan), fungsi *Respond* (melalui orkestrasi *playbook* pengayaan *threat intelligence* dan eksekusi karantina berkas maupun pemblokiran domain), serta fungsi *Recover* (melalui fitur pemulihan berkas karantina apabila terkonfirmasi sebagai positif palsu).

2.3 Intrusion Detection and Prevention System (IDS/IPS)

Intrusion Detection System (IDS) adalah sistem keamanan yang memantau lalu lintas jaringan atau aktivitas di dalam perangkat (*host*) untuk mendeteksi perilaku mencurigakan maupun pelanggaran kebijakan keamanan, kemudian mengirimkan peringatan kepada administrator (Antara & Rachmawati, 2024). Sementara itu, *Intrusion Prevention System* (IPS) memperluas kapabilitas IDS dengan kemampuan mengambil tindakan pencegahan aktif secara langsung untuk menghentikan ancaman yang terdeteksi. Berdasarkan metode deteksinya, IDS diklasifikasikan ke dalam dua pendekatan utama sebagai berikut.

1. ***Signature-based Detection*:** mengidentifikasi ancaman dengan mencocokkan atribut aktivitas atau berkas (seperti pola *log*, *hash* kriptografis SHA-256, atau tanda tangan digital) terhadap basis data indikator serangan yang telah dikenal (*known indicators of compromise*).
2. ***Anomaly/Behavior-based Detection*:** mendeteksi penyimpangan dari perilaku normal sistem, seperti pemanggilan berantai *interpreter* skrip dari direktori sementara atau perubahan mendadak pada berkas umpan (*canary file*).

Berdasarkan lokasi penempatannya, sistem ini dibagi menjadi *Network-based IDS* (NIDS) yang memantau paket jaringan dan *Host-based IDS* (HIDS) yang berjalan langsung pada setiap *endpoint* untuk memantau sistem berkas, proses, serta *log* sistem operasi (Prasetyo et al., 2023). Pada penelitian ini, Wazuh dan agen pemantau ringan berperan sebagai HIDS yang menggabungkan deteksi berbasis *signature* (reputasi *hash* dan URL) serta deteksi berbasis perilaku (aturan rantai proses dan berkas umpan), sedangkan fungsi IPS diwujudkan melalui mekanisme *Active Response* yang dikendalikan secara berjenjang oleh lapisan SOAR.

2.4 Security Information and Event Management (SIEM)

Security Information and Event Management (SIEM) adalah platform keamanan terpusat yang menggabungkan fungsi *Security Information Management* (SIM) untuk penyimpanan dan analisis historis *log*, serta *Security Event Management* (SEM) untuk pemantauan dan korelasi kejadian keamanan secara *real-time* (Heluka & Sulistyo, 2023). SIEM bekerja dengan mengumpulkan data telemetri dari berbagai *endpoint*, server, dan aplikasi, melakukan normalisasi (*decoding*) ke dalam format standar, lalu mencocokkannya dengan aturan korelasi (*correlation rules*) guna menghasilkan *alert* berjenjang (Alves et al., 2026).

Meskipun SIEM sangat efektif sebagai pusat visibilitas dan deteksi awal, arsitektur SIEM tradisional memiliki keterbatasan mendasar pada tahap pasca-deteksi. Peringatan yang dihasilkan oleh aturan SIEM (seperti perubahan berkas pada *File Integrity Monitoring*) umumnya belum memiliki konteks reputasi eksternal, sehingga menghasilkan volume *alert* yang sangat tinggi dan memicu *alert fatigue* pada analis (Tariq et al., 2025). Untuk mengubah *alert* mentah dari SIEM menjadi keputusan mitigasi yang akurat dan terukur, diperlukan lapisan orkestrasi dan otomatisasi yang disediakan oleh sistem SOAR.

2.5 Wazuh sebagai Platform SIEM dan HIDS

Wazuh merupakan platform keamanan sumber terbuka (*open-source*) yang memadukan kapabilitas SIEM dan *Host-based Intrusion Detection System* (HIDS) / *Endpoint Detection and Response* (EDR) (Wazuh Inc., 2026). Arsitektur standar Wazuh terdiri atas tiga komponen inti sebagai berikut.

1. **Wazuh Agent:** perangkat lunak yang berjalan pada *endpoint* untuk mengumpulkan *log* sistem operasi, memantau integritas berkas (*File Integrity Monitoring* / FIM via `wazuh-syscheckd`), memantau peristiwa proses (seperti Windows Sysmon), serta mengeksekusi skrip penanggulangan lokal melalui daemon `wazuh-execd`.
2. **Wazuh Manager:** komponen server pusat yang menerima telemetri terenkripsi dari agen (melalui port TCP 1514), melakukan *decoding* dan pencocokan aturan melalui `wazuh-analysisd`, meneruskan *alert* ke layanan eksternal melalui daemon `wazuh-integratord`, serta menyediakan RESTful API (port 55000) untuk pengelolaan agen dan pemicuan *Active Response*.
3. **Wazuh Indexer dan Dashboard:** mesin pengindeks berbasis OpenSearch yang menyimpan *alert* secara persisten (indeks `wazuh-alerts-*`) untuk pencarian cepat, agregasi statistik, dan visualisasi insiden.

Di samping penggunaan Wazuh Agent standar yang membutuhkan memori residen sekitar 80 hingga 150 MB, lingkungan usaha kecil dan menengah sering kali mengoperasikan banyak komputer kasir atau *workstation* lama dengan kapasitas RAM terbatas. Untuk menjawab tantangan efisiensi pada skenario pemantauan puluhan hingga ratusan *workstation*, penelitian ini juga merancang alternatif **Agen Pemantau Ringan berbasis bahasa pemrograman Rust (`soar-agent`)**. Agen mandiri ini dikompilasi menjadi satu berkas biner statis berukuran sekitar 4 hingga 5 MB dengan konsumsi memori residen di bawah 10 MB, namun tetap kompatibel dengan skema JSON *alert* Wazuh sehingga dapat langsung diproses oleh *pipeline* orkestrasi n8n yang sama.

2.6 Security Orchestration, Automation, and Response (SOAR)

Security Orchestration, Automation, and Response (SOAR) didefinisikan oleh Gartner sebagai arsitektur perangkat lunak yang memungkinkan organisasi mengumpulkan masukan data keamanan dari berbagai komponen, lalu menjalankan prosedur analisis dan penanggulangan

insiden melalui alur kerja terstandarisasi (*playbook*) yang memadukan otomatisasi mesin dan intervensi manusia (Gartner, 2023; Nadig & Vignesh, 2026). Sistem SOAR bertumpu pada tiga pilar kapabilitas utama sebagai berikut.

1. ***Security Orchestration*** **(Orkestrasi):** mengintegrasikan berbagai perkakas keamanan yang terpisah (SIEM, layanan *threat intelligence* multi-sumber, modul AI, basis data *audit trail*, dan kanal komunikasi analis) agar dapat saling bertukar data dalam satu alur kerja terpadu.
2. ***Security Automation*** **(Otomatisasi):** mengeksekusi tugas-tugas analitik yang berulang secara otomatis oleh mesin, meliputi penyaringan kebisingan (*noise filtering*), deduplikasi *alert*, pemeriksaan reputasi *hash* atau URL ke berbagai API, serta penentuan tingkat keparahan.
3. ***Incident Response*** **(Respons Insiden):** mengelola siklus tindakan penanggulangan ancaman, mulai dari pengiriman peringatan kontekstual, eksekusi isolasi berkas atau pemblokiran domain, pencatatan jejak audit keputusan analis, hingga eskalasi batas waktu penanganan.

Melalui penerapan SOAR, waktu rata-rata yang dibutuhkan sejak ancaman terdeteksi hingga berhasil ditanggulangi (*Mean Time to Respond* / MTTR) dapat ditekan dari hitungan menit atau jam menjadi hitungan detik, sekaligus memastikan setiap insiden ditangani secara konsisten sesuai prosedur standar organisasi (Thakker et al., 2025; Nadig & Vignesh, 2026).

2.7 Deteksi Malware dan File Integrity Monitoring

Malware (*malicious software*) mencakup berbagai jenis perangkat lunak berbahaya seperti *trojan*, *ransomware*, *worm*, dan *backdoor* yang dirancang untuk menyusup serta merusak sistem komputasi. Pada tingkat *endpoint*, deteksi *malware* dalam penelitian ini dibangun secara berlapis melalui empat mekanisme sensor yang dipetakan terhadap matriks taktik dan teknik **MITRE ATT&CK Enterprise v15.1** (MITRE, 2026) sebagai berikut.

1. ***File Integrity Monitoring*** **(FIM) dan Reputasi Hash:** memantau pembuatan dan modifikasi berkas secara *real-time* (menggunakan subsistem *inotify* pada Linux dan *ReadDirectoryChangesW* pada Windows) pada direktori bernilai tinggi seperti folder unduhan (Downloads), Desktop, Documents, direktori sistem (/etc/cron.*, /etc/systemd/system, /usr/local/bin), serta media penyimpanan eksternal

(USB/flashdisk) yang dipasang secara dinamis. Setiap berkas baru dihitung nilai *hash* SHA-256-nya tanpa perlu mengirimkan isi berkas utuh ke server.

2. **Deteksi Karakteristik Berkas Berisiko (*Magic-Byte*, Bit Eksekusi, dan Ekstensi Ganda):** untuk mengantisipasi *malware* baru (*zero-day*) yang nilai *hash*-nya belum terdaftar di basis data publik, sensor memeriksa atribut izin eksekusi (*execute bit*), berkas biner tanpa ekstensi, serta teknik penyamaran ekstensi ganda (*Masquerading: Double Extension*, MITRE **T1036.007**) seperti berkas `.pdf.exe` atau `.docx.scr` yang sering digunakan pada serangan *spearphishing*.
3. **Deteksi *Ransomware* Berbasis Berkas Umpan (*Canary File*):** menempatkan berkas umpan tersembunyi pada direktori yang dipantau untuk mendeteksi aktivitas enkripsi massal (*Data Encrypted for Impact*, MITRE **T1486**). Apabila *ransomware* memodifikasi atau mengenkripsi berkas umpan tersebut, sistem segera memicu peringatan darurat berlevel kritis.
4. **Deteksi Rantai Proses *Living-off-the-Land Binaries* (LOLBin):** memantau telemetri pembuatan proses (Sysmon Event ID 1) untuk menangkap serangan yang menyalahgunakan program bawaan sistem operasi yang sah (*Command and Scripting Interpreter* **T1059** dan *System Binary Proxy Execution* **T1218**), seperti pemanggilan berantai `cmd.exe` menuju `powershell.exe` tersembunyi (`-WindowStyle Hidden`) atau penyalahgunaan `rundll32.exe` dan `regsvr32.exe` untuk memuat muatan berbahaya dari direktori sementara.

2.8 Deteksi Phishing

Phishing merupakan teknik rekayasa sosial yang memanipulasi pengguna agar mengakses tautan web palsu guna mencuri kredensial atau mengunduh muatan berbahaya (*Spearphishing Link*, MITRE **T1566.002**, dan *Drive-by Compromise*, **T1189**). Dalam penelitian ini, deteksi *phishing* dirancang melalui dua pendekatan yang saling melengkapi sebagai berikut.

1. **Deteksi *Phishing* Reaktif:** memantau *log* akses URL pada *endpoint* serta mengekstraksi tautan dari berkas pintasan internet (`.url`, `.lnk`, `.desktop`, dan `.webloc`) yang jatuh ke direktori pengguna. Ketika aturan Wazuh atau sensor agen mendeteksi akses ke tautan mencurigakan, URL tersebut diteruskan ke *pipeline* n8n untuk diperiksa reputasinya terhadap Google Safe Browsing (GSB) dan URLScan.io.

2. **Deteksi *Phishing* Proaktif (*Threat Hunting Feed*):** menjalankan alur kerja terjadwal yang secara berkala menarik daftar URL berbahaya aktif terbaru dari umpan publik **URLhaus**, memverifikasinya terhadap Google Safe Browsing, dan secara preventif memicu pemblokiran domain (*sinkhole* ke `0.0.0.0` pada berkas `/etc/hosts`) sebelum tautan tersebut sempat diakses oleh pengguna di lingkungan organisasi.

2.9 Orkestrasi Workflow dengan n8n

n8n adalah platform otomatisasi dan orkestrasi alur kerja (*workflow automation*) sumber terbuka berbasis *node* yang memungkinkan pengintegrasian berbagai layanan melalui antarmuka visual maupun penulisan kode JavaScript/Python kustom (n8n GmbH, 2026). Dibandingkan dengan platform SOAR lain seperti Shuffle, n8n menawarkan fleksibilitas manipulasi struktur data JSON yang tinggi pada *Code node*, kemudahan pengelolaan kredensial terpusat, jejak eksekusi per-*node* yang transparan, serta penggunaan memori yang jauh lebih ringan untuk dijalankan di dalam kontainer Docker pada perangkat server berspesifikasi rendah.

Dalam mengimplementasikan platform orkestrasi keamanan, aspek keamanan dari mesin SOAR itu sendiri wajib diperhatikan. Laporan intelijen ancaman tahun 2026 menyoroti adanya penyalahgunaan platform otomasi serta kerentanan kritis pada versi lama n8n, seperti CVE-2026-21858 (“Ni8mare”) dan batch CVE Oktober 2026 pada versi sebelum 2.40.1 yang memungkinkan eksekusi kode jarak jauh atau pembacaan berkas melalui *webhook* yang terekspos ke publik (Cisco Talos, 2026). Oleh karena itu, sistem pada penelitian ini menerapkan prinsip pengerasan berlapis (*hardening*): menggunakan versi n8n ter-patch terbaru (`2.42.5`), tidak mengekspos *webhook* n8n ke jaringan internet publik, serta menggunakan mekanisme penjemputan pesan keluar (*outbound-only long-polling*) untuk interaksi dengan Telegram Bot API.

2.10 Threat Intelligence dan Analisis LLM

Threat Intelligence adalah informasi berbasis bukti mengenai indikator ancaman yang digunakan untuk memverifikasi apakah suatu *alert* SIEM merupakan serangan nyata atau sekadar aktivitas administratif biasa. Mengandalkan satu sumber intelijen tunggal seperti VirusTotal memiliki keterbatasan berupa batas kuota permintaan (*rate limit* 4 permintaan/menit pada *tier* gratis), potensi keterlambatan pembaruan (*detection lag*), serta kebutaan terhadap

sampel yang baru diunggah ke repositori spesifik. Oleh sebab itu, penelitian ini menerapkan arsitektur **koroborasi *threat intelligence* multi-sumber** sebagai berikut.

1. **Jalur Verifikasi Malware (Hash SHA-256):** menggabungkan **VirusTotal API v3** (konsensus 70+ mesin antivirus), **MalwareBazaar (abuse.ch)** (basis data sampel *malware* aktif), **AlienVault OTX** (sebagai jalur *fallback* otomatis ketika VirusTotal mengalami *rate-limit* atau gangguan), serta layanan indeks lokal **MISP OSINT Feed** (misp-index yang menyinkronkan ribuan *event* indikator kompromi dari CIRCL OSINT tanpa batas kuota API). Hasil pemeriksaan disimpan pada *cache* sisi server dengan masa berlaku (*Time-to-Live* / TTL) diferensial, yaitu 7 hari untuk berkas berbahaya, 24 jam untuk berkas bersih, dan 6 jam untuk berkas yang belum dikenal.
2. **Jalur Verifikasi Phishing (URL dan Domain):** memadukan **Google Safe Browsing API v4** untuk pemeriksaan daftar hitam tautan penipuan/*malware* secara cepat, **URLScan.io API** untuk pemindaian perilaku halaman web, serta umpan **URLhaus** untuk pencegahan proaktif.

Setelah bukti *threat intelligence* terkumpul, sistem memanfaatkan **Large Language Model (LLM)** melalui API penyedia (DeepSeek-V4-Flash dan Gemini 2.5 Flash) untuk menghasilkan ringkasan insiden dan rekomendasi tindakan dalam Bahasa Indonesia. Selaras dengan temuan riset mengenai risiko halusinasi AI dan *prompt injection* pada aplikasi agen keamanan (Hammar et al., 2025; Srinivas et al., 2025; OWASP, 2026), modul LLM pada penelitian ini dirancang dengan tiga pembatas keamanan (*guardrail*) utama: (1) menerapkan teknik *Retrieval-Augmented Generation* (RAG) ringan yang menyuntikkan dokumen *playbook* penanganan lokal sesuai tingkat keparahan ke dalam *prompt*, (2) menginstruksikan model untuk memprioritaskan fakta indikator (*Evidence-First*) dan mengenali aktivitas sah sistem seperti penulisan *sinkhole* pada berkas hosts, serta (3) menempatkan LLM murni sebagai pemberi rekomendasi (*advisory*) tanpa akses untuk memicu *Active Response* secara langsung.

2.11 Active Response Human-in-the-Loop dan Integrasi Telegram Bot

Integrasi sistem deteksi dengan platform pesan instan **Telegram Bot** menyediakan saluran komunikasi *real-time* yang cepat, ringan, dan mudah diakses oleh administrator dari mana saja melalui Telegram Bot API (Telegram, 2026). Dalam penelitian ini, Telegram Bot tidak hanya berfungsi sebagai penampil peringatan satu arah, melainkan dirancang sebagai

antarmuka kendali dua arah (*bidirectional command interface*) melalui komponen *inline keyboard*.

Landasan pengambilan keputusan respons pada sistem ini mengadopsi paradigma ***Trusted Autonomy*** dan **Active Response *human-in-the-loop* (HITL)** (Mohsin et al., 2025; Singh et al., 2025). Alih-alih mengeksekusi tindakan destruktif secara membabi buta pada setiap *alert* perubahan berkas, sistem membagi otonomi tindakan ke dalam jenjang berbasis keyakinan bukti sebagai berikut.

1. **Otonomi Penuh pada Keyakinan Tinggi (*Auto-Response*):** apabila konsensus *threat intelligence* membuktikan berkas atau tautan positif berbahaya secara meyakinkan (misalnya terdeteksi oleh sedikitnya 5 hingga 20 mesin antivirus di VirusTotal atau terdaftar pada MalwareBazaar/MISP/GSB), sistem langsung mengeksekusi tindakan penanggulangan otomatis (`quarantine-file` atau `block-domain`) untuk meminimalkan jeda paparan, sembari tetap menyediakan tombol umpan balik bagi analis untuk menandai positif palsu atau memulihkan berkas.
2. **Kendali Manusia pada Keyakinan Sedang atau Indikasi Perilaku (*Human-in-the-Loop*):** apabila berkas memiliki indikasi lemah (1 hingga 4 deteksi), gagal diverifikasi akibat batas kuota API (*degraded state*), merupakan berkas eksekutabel dengan *hash* yang belum dikenal (*zero-day candidate*), atau merupakan rantai proses sistem operasi (LOLBin) yang berisiko merusak sistem jika dihentikan secara paksa, sistem menahan eksekusi otomatis dan mengirimkan notifikasi interaktif dilengkapi tombol aksi (`Karantina file`, `Sudah ditinjau`, atau `Tandai FP`).
3. **Akuntabilitas Audit dan Eskalasi SLA:** setiap insiden yang membutuhkan keputusan analis didaftarkan ke dalam layanan *registry* keputusan persisten (`alert-log`) dengan batas waktu tanggap (*Service Level Agreement* / SLA) selama 15 menit (900 detik). Apabila analis belum memberikan keputusan hingga batas waktu terlampaui, alur kerja pemantau SLA secara otomatis mengirimkan peringatan eskalasi ke Telegram agar tidak ada insiden kritis yang terabaikan diam-diam. Seluruh keputusan yang diambil analis tercatat lengkap beserta identitas pengambil keputusan, stempel waktu, dan durasi respons sebagai jejak audit (*audit trail*) forensik.

BAB III
METODOLOGI PENELITIAN

Bab III memuat metode yang digunakan dalam penelitian "Implementasi Sistem SOAR *Open-Source* Berbasis n8n untuk Deteksi dan Respons Ancaman Malware dan Phishing dengan Mitigasi Aktif *Human-in-the-Loop* (Studi Kasus: CV Bali Handmade)". Bagian metodologi penelitian membahas mengenai waktu dan tempat penelitian, sumber data yang digunakan, instrumen perancangan sistem, tahapan penelitian, metode pengembangan sistem, desain arsitektur, perancangan *pipeline* deteksi dan klasifikasi *severity*, desain *Active Response human-in-the-loop* dengan integrasi Telegram Bot, serta perancangan modul riwayat dan *monitoring event*.

3.1 Waktu dan Tempat Penelitian

Penelitian ini dilaksanakan mulai dari bulan September 2025 hingga Oktober 2026. Penelitian Tugas Akhir dilaksanakan di Kampus Bukit Universitas Udayana, Fakultas Teknik, Program Studi Teknologi Informasi yang beralamat di Jalan Kampus Bukit UNUD, Jimbaran, Kuta Selatan, Badung, Bali, serta di Perpustakaan Universitas Udayana dan lokasi pengujian mandiri penulis yang berlokasi di Pasekan, Tabanan, Bali, dengan studi kasus kebutuhan pemantauan keamanan pada CV Bali Handmade.

3.2 Sumber Data

Pengumpulan data dalam penelitian ini dilakukan melalui kombinasi data primer hasil pengujian sistem secara langsung dan data sekunder dari studi literatur serta layanan intelijen ancaman eksternal. Rincian sumber data yang digunakan adalah sebagai berikut.

1. **Literatur Ilmiah Terkini:** artikel jurnal nasional dan internasional pada rentang tahun 2020 hingga 2026 yang membahas penerapan SIEM Wazuh, orkestrasi SOAR berbasis n8n dan Shuffle, koroborasi *threat intelligence*, pemanfaatan *Large Language Model* (LLM) untuk triase insiden, paradigma *Trusted Autonomy*, serta *Active Response human-in-the-loop*.
2. **Dokumentasi Teknis dan Basis Pengetahuan Standar:** dokumentasi resmi Wazuh v4.14, n8n Workflow Automation v2.42, VirusTotal API v3, MalwareBazaar API, AlienVault OTX, Google Safe Browsing API v4, URLScan.io, URLhaus, CIRCL MISP

OSINT Feed, Telegram Bot API, NIST SP 800-61 Rev. 3, OWASP Top 10 for LLM Applications, serta matriks taktik dan teknik MITRE ATT&CK Enterprise.

3. **Data Uji Telemetri dan Simulasi Serangan:** data *log* dan *alert* primer yang dihasilkan dari simulasi terkendali pada *endpoint* lintas sistem operasi, meliputi berkas uji standar aman **EICAR** (*European Institute for Computer Antivirus Research*), modifikasi berkas umpan *ransomware* (*canary file*), simulasi penyamaran ekstensi ganda (*double extension*), eksekusi rantai proses *Living-off-the-Land Binaries* (LOLBin) pada Windows Sysmon, serta simulasi akses tautan *phishing* reaktif dan penarikan umpan *phishing* proaktif.

3.3 Instrumen Perancangan Sistem

Dalam merancang dan mengimplementasikan sistem pada penelitian ini, diperlukan instrumen pendukung yang terbagi ke dalam dua komponen utama, yaitu perangkat keras (*hardware*) dan perangkat lunak (*software*).

3.3.1 Perangkat Keras (Hardware)

Perangkat keras yang digunakan terdiri atas satu laptop utama yang berperan sebagai **SOAR Server** (menjalankan kontainer Wazuh Manager, Indexer, Dashboard, n8n, layanan mikro alert-log dan misp-index, serta dasbor fleet-monitor) sekaligus *endpoint* pertama, serta perangkat *endpoint* kedua dan mesin virtual pengujian lintas sistem operasi. Spesifikasi perangkat keras yang digunakan disajikan pada Tabel 3.1.

Tabel 3.1 Spesifikasi Perangkat Keras

Perangkat	Peran	Spesifikasi
Laptop Utama (ravi-zorin)	SOAR Server + Endpoint Agent #1	CPU Intel Core i5-1235U, RAM 16 GB, Penyimpanan 512 GB NVMe SSD, GPU NVIDIA MX 550, OS Ubuntu/Zorin Linux
ThinkPad X260 (rocky-server)	Endpoint Agent #2 (Server Uji)	CPU Intel Core i5, RAM 8 GB, Penyimpanan 256 GB SSD, OS Rocky Linux 9
Workstation Virtual (win10-endpoint)	Endpoint Agent #3 (Uji Lintas-OS)	Alokasi 2 vCPU, RAM 4 GB, OS Windows 10 Pro (Sysmon + Wazuh Agent + soar-agent)

Pemilihan kombinasi *endpoint* dengan distribusi Linux berbeda (keluarga Debian/Ubuntu dan RHEL/Rocky Linux) serta sistem operasi Windows bertujuan untuk memvalidasi kemampuan deteksi, orkestrasi, dan *Active Response* secara lintas platform (*cross-platform multi-endpoint*).

3.3.2 Perangkat Lunak (Software)

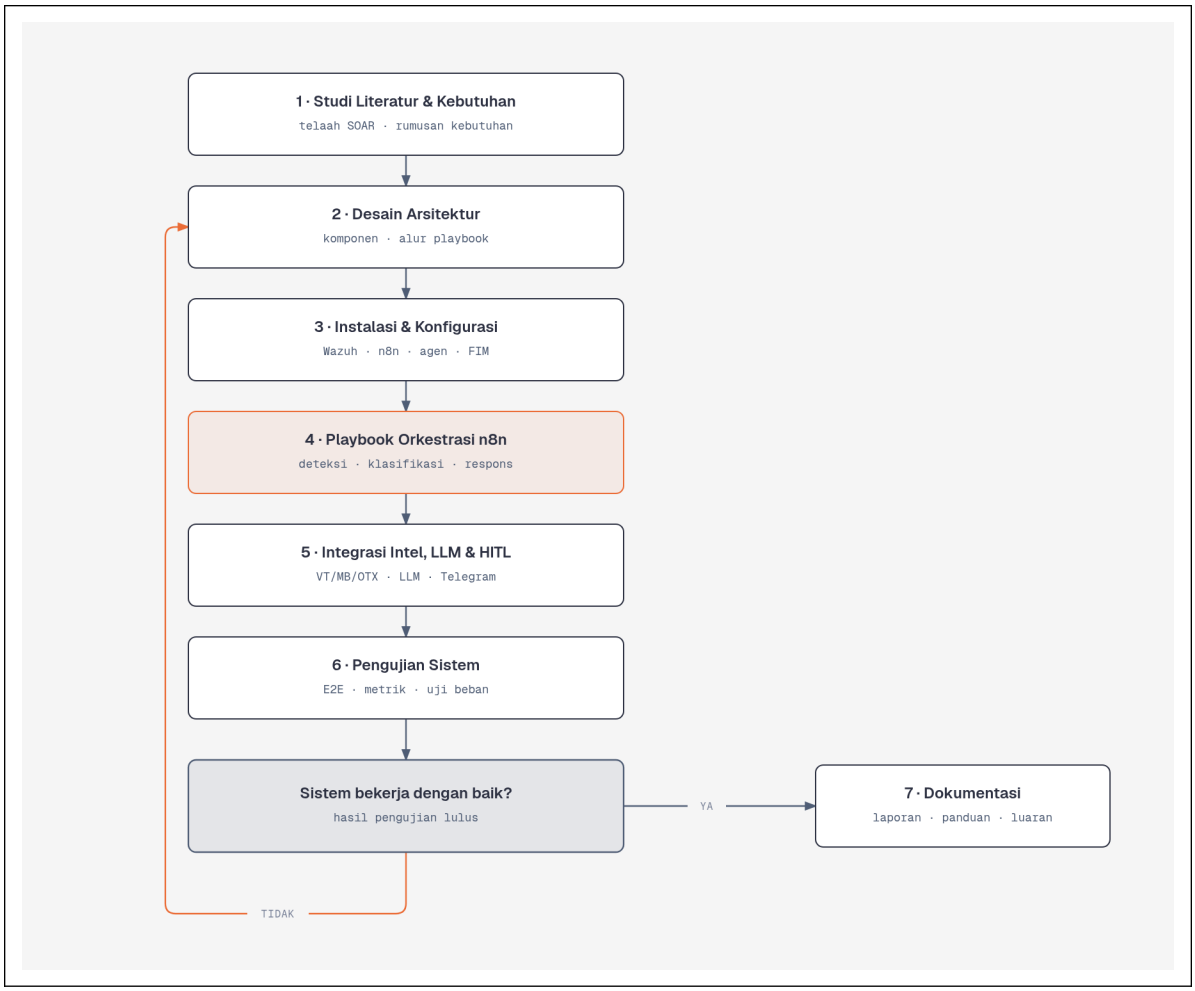
Perangkat lunak yang digunakan dalam pembangunan sistem ini seluruhnya berbasis sumber terbuka (*open-source*) atau memanfaatkan layanan API pada *tier* komunitas/gratis. Rincian perangkat lunak beserta versi terverifikasi disajikan pada Tabel 3.2.

Tabel 3.2 Daftar Perangkat Lunak

Perangkat Lunak	Versi	Peran dalam Sistem
Wazuh Manager, Indexer, Dashboard, & Agent	4.14.8	Platform SIEM/HIDS utama, korelasi aturan deteksi, penyimpanan indeks OpenSearch (`wazuh-alerts-*`), dan eksekutor *Active Response*
n8n Workflow Automation	2.42.5	Mesin orkestrasi SOAR (menjalankan 5 *workflow* aktif: Triase Malware, Triase Phishing, HITL Callback, Threat Feed, dan Pemantau SLA/Health)
Docker & Docker Compose	27.x / v2	Kontainerisasi dan isolasi jaringan layanan SOAR Server
`soar-agent` (Rust)	0.3.0	Agen pemantau *endpoint* mandiri lintas-OS berukuran ringan (~5 MB) dengan antarmuka GUI (`egui`) dan sensor FIM/YARA/Canary
`fleet-monitor` & Dashboard Web	Python 3.12 / Next.js	Layanan agregator status armada *endpoint*, penyimpanan riwayat *event* SQLite, antarmuka web, dan terminal TUI
`alert-log` & `misp-index`	Python 3.12 (SQLite)	Layanan *registry* keputusan analis ber-SLA 15 menit dan indeks lokal ribuan indikator kompromi CIRCL MISP OSINT Feed
Threat Intelligence API Multi-Sumber	REST API v3 / v4	VirusTotal API v3, MalwareBazaar, AlienVault OTX (untuk *hash*), serta Google Safe Browsing v4, URLScan.io, dan URLhaus (untuk URL)
API Large Language Model (LLM)	DeepSeek-V4 / Gemini 2.5	Penyusunan ringkasan insiden dan rekomendasi tindakan berbahasa Indonesia berbasis RAG *playbook* lokal
Telegram Bot API & `tg-callback-poller`	Bot API 7.x / Python 3.12	Kanal notifikasi *real-time* dua arah (*inline keyboard*) dan penjemput klik tombol melalui *long-polling* keluar
Tailscale & Cloudflare Access	1.98.x	Jaringan *mesh* VPN terenkripsi antar-*endpoint* dan pembatas akses *zero-trust* untuk *dashboard*

3.4 Tahapan Penelitian

Tahapan penelitian menjelaskan langkah-langkah sistematis yang dilakukan dalam merancang, membangun, dan mengevaluasi sistem SOAR *open-source* berbasis n8n. Alur tahapan penelitian secara keseluruhan ditunjukkan pada Gambar 3.1.



Gambar 3.1 Tahapan Penelitian

Sebagaimana ditunjukkan pada Gambar 3.1, pelaksanaan penelitian ini terbagi ke dalam sembilan tahapan berurutan sebagai berikut.

1. **Studi Literatur dan Analisis Kebutuhan:** mengkaji referensi jurnal ilmiah terkini (2020-2026) serta memetakan kebutuhan operasional pemantauan keamanan siber pada studi kasus CV Bali Handmade, termasuk keterbatasan perangkat keras dan kebutuhan kendali manusia atas tindakan kritis.
2. **Desain Arsitektur Sistem:** merancang topologi jaringan berlapis yang menghubungkan *endpoint* multi-OS, Wazuh Stack, mesin orkestrasi n8n, layanan mikro pendukung (alert-log, misp-index, fleet-monitor), *threat intelligence* multi-sumber, LLM, dan Telegram Bot.

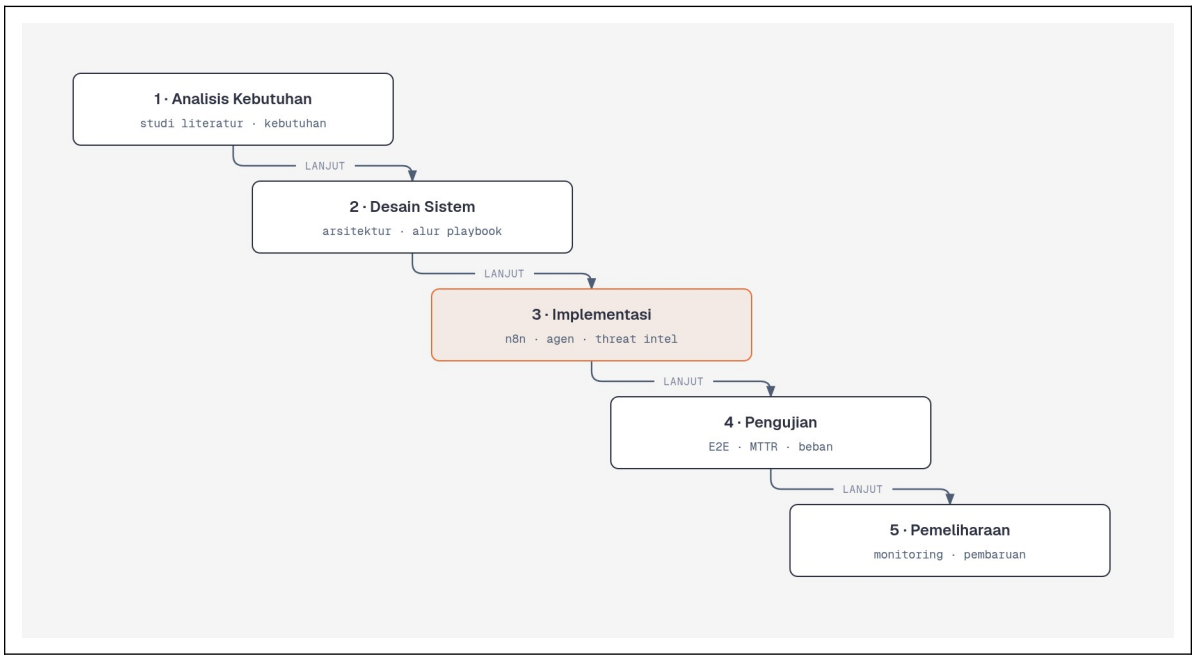
3. **Instalasi dan Konfigurasi Infrastruktur:** menerapkan kontainerisasi Wazuh 4.14.8 dan n8n 2.42.5 menggunakan Docker Compose, mengonfigurasi konektivitas Tailscale mesh VPN, serta memasang Wazuh Agent dan soar-agent pada *endpoint* uji.
4. **Konfigurasi Sensor Deteksi (FIM, YARA, Canary, dan Rantai Proses):** menyusun aturan deteksi kustom pada Wazuh dan sensor agen yang dipetakan terhadap kerangka MITRE ATT&CK, mencakup pemantauan direktori bernilai tinggi, deteksi ekstensi ganda, berkas umpan *ransomware*, rantai proses LOLBin, serta *log* akses tautan *phishing*.
5. **Pembangunan Playbook Orkestrasi n8n dan Matriks Otonomi:** membangun lima *workflow* otomatis di n8n serta mengimplementasikan logika node tunggal Tentukan Otonomi yang memetakan kekuatan bukti ancaman ke enam tingkat otonomi tindakan.
6. **Integrasi Threat Intelligence Multi-Sumber dan Analisis LLM:** menghubungkan *playbook* dengan VirusTotal, MalwareBazaar, AlienVault OTX, MISP lokal, Google Safe Browsing, URLScan.io, dan URLhaus lengkap dengan *cache* ber-TTL diferensial, serta mengintegrasikan analisis kontekstual LLM berbasis RAG *playbook* lokal.
7. **Integrasi Telegram Bot, Active Response HITL, dan SLA Registry:** membangun antarmuka keputusan dua arah melalui *inline keyboard* Telegram, layanan *long-polling* tg-callback-poller, skrip quarantine-file (beserta fitur *restore*) dan block-domain, serta pemantauan tenggat SLA 15 menit pada alert-log.
8. **Pengujian Skenario Serangan dan Evaluasi Kinerja:** menjalankan pengujian fungsionalitas dan performa melalui skenario serangan terkendali (*malware*, *ransomware canary*, rantai proses, *phishing* reaktif/proaktif, degradasi API, dan uji konsistensi LLM) serta mengukur metrik MTTD, MTTR, akurasi klasifikasi, dan reduksi kebisingan.
9. **Penyusunan Dokumentasi dan Laporan:** mendokumentasikan arsitektur, hasil eksperimen, analisis kesenjangan, serta kesimpulan ke dalam naskah laporan Tugas Akhir.

3.5 Metode Pengembangan Sistem

Metode pengembangan perangkat lunak yang digunakan dalam penelitian ini adalah model **Waterfall** (Pressman & Maxim, 2020). Model ini dipilih karena menyediakan alur rekayasa yang terstruktur dan berurutan, di mana setiap fase diselesaikan dan diverifikasi

sebelum melangkah ke fase berikutnya. Lima tahapan utama dalam metode Waterfall adalah sebagai berikut.

1. Analisis Kebutuhan (*Requirement Analysis*)
2. Desain Sistem (*System Design*)
3. Implementasi (*Implementation*)
4. Pengujian (*Testing*)
5. Pemeliharaan (*Maintenance*)



Gambar 3.2 Tahapan Model Waterfall

Gambar 3.2 memperlihatkan kelima fase model Waterfall yang diterapkan dalam proyek penelitian ini. Penjabaran aktivitas pada masing-masing fase diuraikan pada subbab berikut.

3.5.1 Analisis Kebutuhan (Requirement Analysis)

Tahap ini mengidentifikasi kebutuhan fungsional dan nonfungsional dari sistem SOAR berdasarkan studi literatur serta observasi kebutuhan studi kasus CV Bali Handmade. Kebutuhan fungsional sistem meliputi delapan poin utama sebagai berikut.

1. Sistem mampu mengumpulkan dan menganalisis telemetri keamanan dari *endpoint* lintas sistem operasi menggunakan Wazuh maupun agen pemantau ringan `soar-agent`.

2. Sistem mampu mendeteksi ancaman *malware* melalui *File Integrity Monitoring* (FIM), aturan YARA, deteksi ekstensi ganda, berkas umpan *ransomware* (*canary*), dan rantai proses LOLBin.
3. Sistem mampu mendeteksi ancaman *phishing* baik secara reaktif (dari *log* akses URL dan berkas pintasan) maupun secara proaktif (melalui penarikan umpan ancaman terjadwal).
4. Sistem mampu mengorkestrasi pengayaan *threat intelligence* multi-sumber dengan mekanisme *cache* ber-TTL diferensial dan *fallback* otomatis saat salah satu API mengalami batas kuota (*rate-limit*).
5. Sistem mampu memetakan tingkat keparahan (*severity*) dan tingkat otonomi tindakan secara deterministik sesuai matriks *confidence-to-autonomy*.
6. Sistem mampu menghasilkan ringkasan analisis kontekstual berbahasa Indonesia menggunakan LLM berbasis *Retrieval-Augmented Generation* (RAG) *playbook* lokal dengan pembatas anti-halusinasi.
7. Sistem mampu mengirimkan notifikasi interaktif ke Telegram Bot dan mengeksekusi tindakan *Active Response* (`quarantine-file`, `restore`, atau `block-domain`) sesuai persetujuan analis (*human-in-the-loop*).
8. Sistem menyediakan pencatatan keputusan ber-SLA 15 menit beserta mekanisme eskalasi otomatis dan jejak audit (*audit trail*) forensik.

Adapun kebutuhan nonfungsional sistem meliputi lima poin sebagai berikut.

1. Seluruh komponen server berjalan secara terisolasi di dalam kontainer Docker pada perangkat keras kelas laptop tanpa memerlukan biaya lisensi perangkat lunak komersial.
2. Waktu deteksi hingga pengiriman notifikasi (*end-to-end*) berlangsung secara *real-time* (target di bawah 30 detik), sedangkan eksekusi *Active Response* pasca-keputusan berlangsung di bawah 5 detik.
3. Mesin orkestrasi n8n tidak mengekspos *port webhook* ke jaringan internet publik dan memanfaatkan *long-polling* keluar guna mencegah risiko eksploitasi dari luar.
4. Sistem tetap beroperasi secara anggun (*graceful degradation*) apabila API *threat intelligence* eksternal maupun layanan LLM mengalami gangguan konektivitas.
5. Agen pemantau mampu menjaga keterhubungan lintas jaringan melalui Tailscale mesh VPN serta memiliki opsi biner ringan untuk *workstation* berkapasitas memori rendah.

3.5.2 Desain Sistem (System Design)

Tahap desain menerjemahkan daftar kebutuhan di atas ke dalam rancangan arsitektur sistem lima lapis, perancangan *pipeline* deteksi dan matriks klasifikasi tingkat otonomi, pemetaan aturan deteksi ke kerangka MITRE ATT&CK, rancangan *sequence diagram* eksekusi *Active Response human-in-the-loop* beserta format notifikasi Telegram Bot, serta rancangan modul riwayat dan *monitoring event* yang dijabarkan secara rinci pada Subbab 3.6 hingga Subbab 3.9.

3.5.3 Implementasi (Implementation)

Tahap implementasi mewujudkan seluruh rancangan ke dalam konfigurasi infrastruktur dan kode program yang berjalan secara terpadu. Langkah-langkah implementasi yang dilakukan adalah sebagai berikut.

1. **Penerapan Infrastruktur Kontainer:** menjalankan *stack* Wazuh 4.14.8 dan n8n 2.42.5 melalui Docker Compose, serta menyiapkan layanan mikro pendukung (alert-log, misp-index, dan fleet-monitor).
2. **Pembangunan Jembatan Integrasi (custom-n8n):** menyusun modul integrasi yang dipanggil oleh wazuh-integratord untuk menyaring kebisingan (*noise filtering* berdasarkan *rule level* dan *path*), melakukan deduplikasi awal, serta mengonversi *alert* Wazuh ke muatan JSON terstruktur menuju *webhook* lokal n8n.
3. **Implementasi Lima Workflow Orkestrasi n8n:** membangun alur kerja otomatis yang mencakup (1) *Workflow Deteksi dan Triase Malware*, (2) *Workflow Deteksi dan Triase Phishing*, (3) *Workflow Telegram Callback Handler (HITL)*, (4) *Workflow Proactive Threat Hunting Feed (URLhaus)*, serta (5) *Workflow Pemantau Kesehatan Sistem dan Eskalasi SLA 15 Menit*.
4. **Implementasi Koroborasi Intel dan RAG LLM:** mengintegrasikan kueri berjenjang ke VirusTotal, MalwareBazaar, AlienVault OTX, MISP lokal, Google Safe Browsing, dan URLScan.io, serta menyusun modul penyuntikan dokumen *playbook* penanganan lokal ke *prompt* LLM.
5. **Pengembangan Skrip Active Response dan Agen Lintas-Platform:** menyusun skrip quarantine-file (dilengkapi penyimpanan metadata .meta dan fungsi *restore*

berverifikasi SHA-256) serta skrip block-domain (*sinkhole* /etc/hosts), layanan tg-callback-poller, dan aplikasi agen mandiri soar-agent berbasis Rust.

3.5.4 Pengujian (Testing)

Tahap pengujian dilakukan untuk memverifikasi bahwa setiap komponen dan alur kerja sistem berjalan sesuai spesifikasi kebutuhan serta mampu mendeteksi dan menanggulangi skenario ancaman secara akurat. Indikator dan parameter pengujian yang digunakan dalam penelitian ini disajikan pada Tabel 3.5.

Tabel 3.5 Tabel Indikator Pengujian

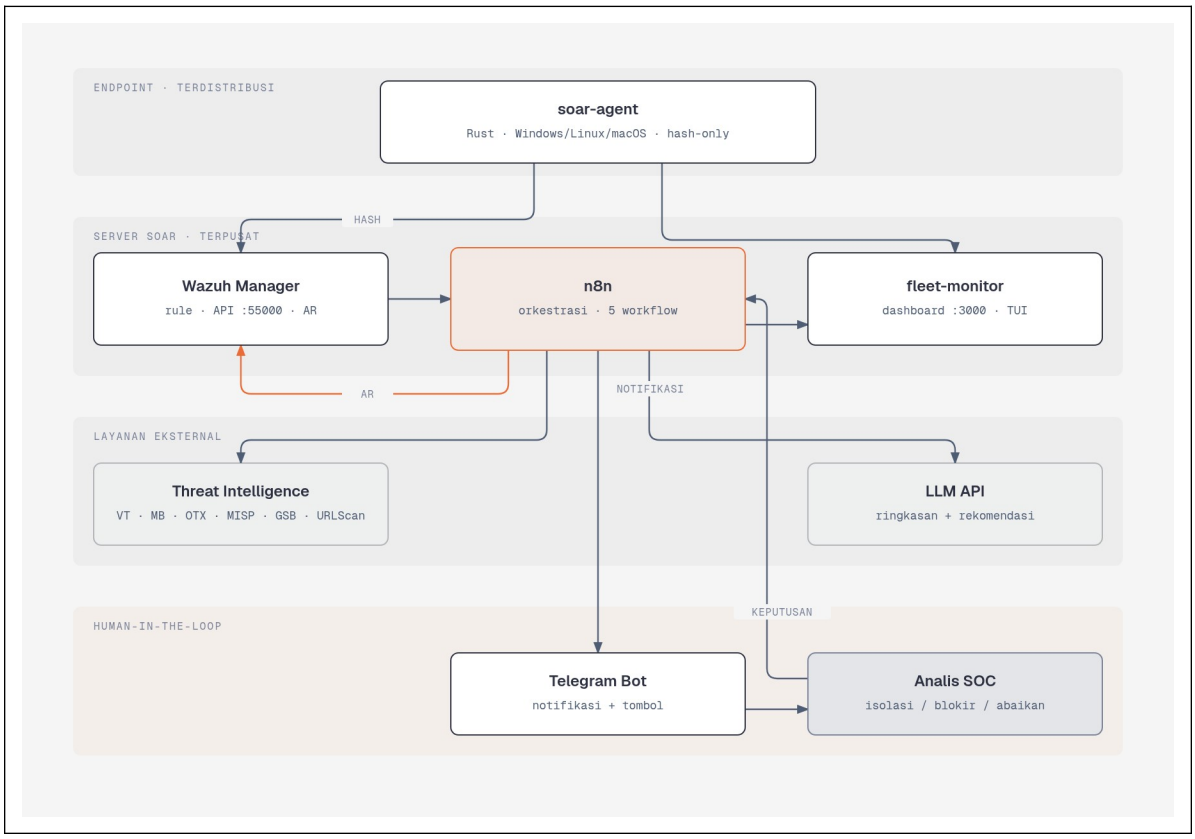
Jenis Pengujian	Tujuan Pengujian	Indikator dan Parameter Keberhasilan
Functional & Scenario Testing	Memastikan seluruh skenario deteksi, pengayaan, dan klasifikasi berjalan sesuai matriks	Keberhasilan deteksi *malware* (EICAR, ekstensi ganda, *canary ransomware*), rantai proses LOLBin, serta *phishing* reaktif dan proaktif
False Positive & Guardrail Testing	Menguji ketahanan sistem terhadap perubahan berkas jinak dan modifikasi berkas sistem	Berkas bersih (VT=0) masuk ke tingkat observe (tanpa karantina); modifikasi /etc/hosts masuk ke manual-config tanpa merusak sistem
API Resilience & Fallback Testing	Menguji mekanisme *fallback* saat salah satu sumber *threat intelligence* gagal atau *rate-limited*	Beralih otomatis ke MalwareBazaar/OTX/MISP lokal; bila seluruh API gagal, *alert* dialihkan dengan aman ke antrean hitl-review
HITL Active Response, Restore & SLA Testing	Menguji eksekusi tombol Telegram, pemulihan berkas, dan eskalasi batas waktu SLA	Karantina/blokir dieksekusi < 5 detik pasca-klik, *restore* mengembalikan berkas secara utuh, dan pengingat SLA terkirim saat lewat 15 menit
Performance & Agent Footprint Testing	Mengukur waktu tanggap *end-to-end*, konsistensi LLM, dan efisiensi sumber daya agen	MTTD, MTTR, akurasi kepatuhan rekomendasi LLM terhadap *playbook*, serta perbandingan penggunaan RAM/CPU antara Wazuh Agent dan soar-agent

3.5.5 Pemeliharaan (Maintenance)

Tahap pemeliharaan mencakup pemantauan kesehatan layanan secara otomatis melalui *workflow* pemeriksaan mandiri setiap 5 menit (memeriksa keterjangkauan Wazuh API, layanan alert-log, misp-index, dan kuota penyedia), pembaruan berkala basis data indikator ancaman lokal dari CIRCL MISP OSINT dan URLhaus, pemangkasan otomatis riwayat *event* SQLite yang melewati masa retensi 90 hari, serta pencadangan (*backup*) berkas definisi *workflow* n8n dan aturan kustom Wazuh.

3.6 Desain Arsitektur Sistem

Desain arsitektur sistem menggambarkan hubungan struktural dan alur komunikasi antar-komponen yang membentuk ekosistem SOAR sumber terbuka. Rancangan arsitektur sistem secara menyeluruh ditunjukkan pada Gambar 3.3.



Gambar 3.3 Desain Arsitektur Sistem SOAR

Sebagaimana terlihat pada Gambar 3.3, sistem dirancang dengan arsitektur **lima lapisan (*five-layer architecture*)** agar setiap komponen memiliki tanggung jawab yang terpisah dan tegas (*high cohesion, low coupling*) sebagai berikut.

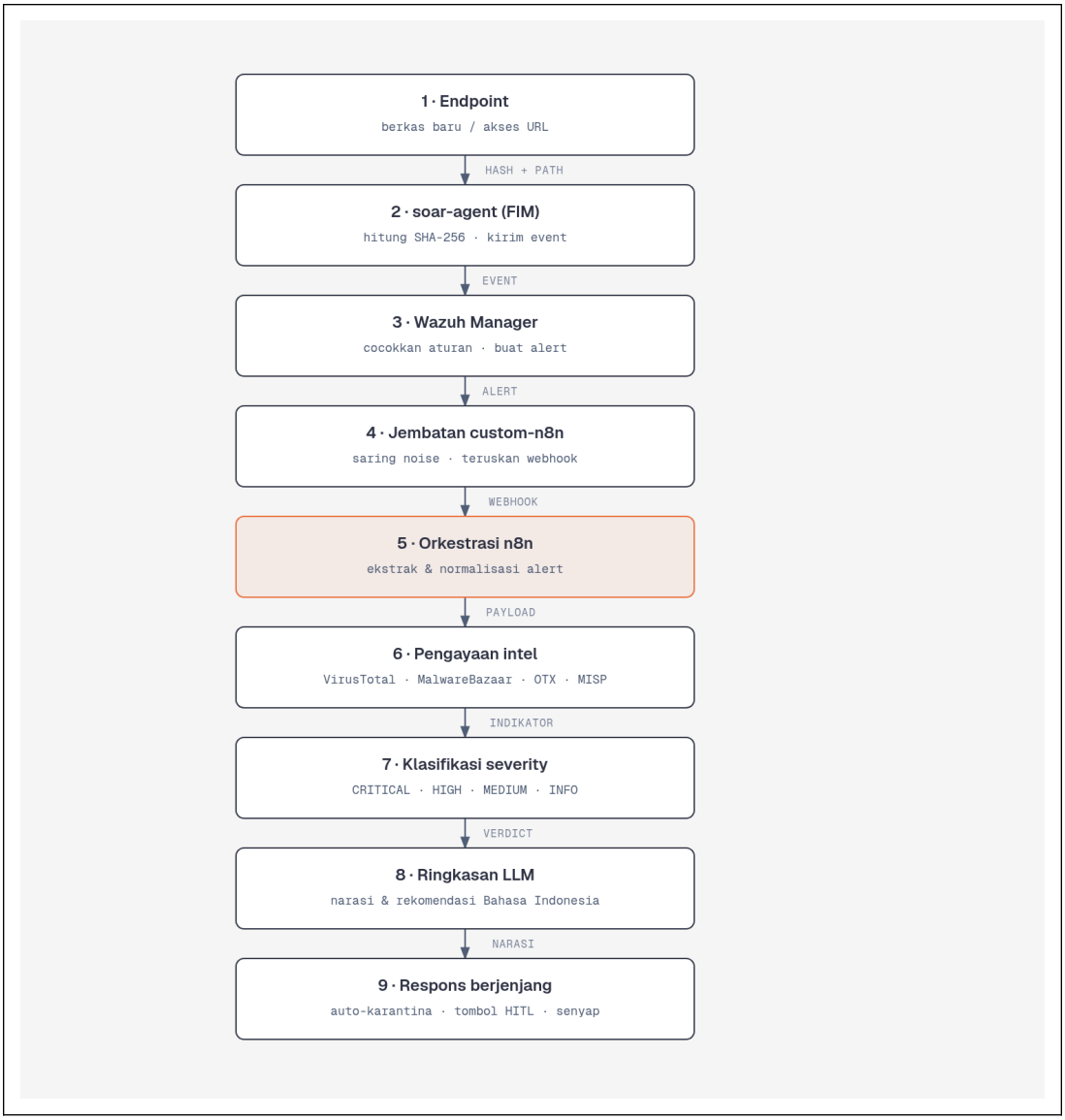
1. **Layer 1: Deteksi Terdistribusi (*Distributed Endpoint Layer*):** berada pada masing-masing *endpoint* (Linux dan Windows) yang menjalankan **Wazuh Agent** maupun agen ringan **soar-agent**. Lapisan ini melakukan pemantauan integritas berkas secara *real-time*, pemindaian karakteristik berkas dan YARA, pemantauan berkas umpan *canary*, serta pengumpulan telemetri rantai proses dan akses URL.
2. **Layer 2: Agregasi dan Korelasi SIEM (*Centralized SIEM Layer*):** berada pada SOAR Server di dalam kontainer **Wazuh Manager** dan **Wazuh Indexer**. Komponen wazuh-

remoted menerima telemetri agen melalui port TCP 1514 terenkripsi, wazuh-analysisd mencocokkan *event* terhadap aturan bawaan maupun aturan kustom (local_rules.xml), dan wazuh-integratord meneruskan *alert* yang memenuhi kriteria ke jembatan integrasi menuju n8n.

3. **Layer 3: Orkestrasi dan Layanan Mikro SOAR (*Centralized Orchestration Layer*):** dijalankan oleh kontainer **n8n** bersama layanan mikro lokal **alert-log** (pencatat siklus hidup keputusan HITL dan tenggat SLA 15 menit), **misp-index** (indeks lokal indikator ancaman OSINT), serta **fleet-monitor** (agregator status armada dan riwayat *event*).
4. **Layer 4: Pengayaan Threat Intelligence dan Analisis AI (*Enrichment & Advisory Layer*):** mengoordinasikan verifikasi reputasi indikator terhadap **VirusTotal API v3**, **MalwareBazaar**, **AlienVault OTX**, **Google Safe Browsing API v4**, **URLScan.io**, dan **URLhaus**, lalu menyusun penjelasan kontekstual serta rekomendasi penanganan menggunakan **LLM API** berbasis dokumen *playbook* lokal.
5. **Layer 5: Kendali Manusia dan Eksekusi Respons (*Human-in-the-Loop Response Layer*):** menghubungkan sistem dengan analis melalui **Telegram Bot** (dijembatani oleh tg-callback-poller) serta mengeksekusi perintah mitigasi balik (quarantine-file, restore, atau block-domain) ke *endpoint* sasaran melalui Wazuh RESTful API maupun saluran perintah agen.

3.7 Perancangan Pipeline Deteksi dan Klasifikasi Severity

Bagian ini menjelaskan proses *pipeline* deteksi dan triase *end-to-end* mulai dari munculnya *event* pada *endpoint*, pengayaan *threat intelligence* berlapis, penentuan tingkat keparahan dan otonomi tindakan, hingga pengiriman notifikasi interaktif. Diagram alir *pipeline* deteksi ditunjukkan pada Gambar 3.4.



Gambar 3.4 Alur Pipeline Deteksi End-to-End

Berdasarkan Gambar 3.4, alur kerja *pipeline* deteksi dan triase berjalan melalui delapan tahap terstruktur sebagai berikut.

1. **Deteksi Sensor pada Endpoint:** saat berkas baru dibuat/dimodifikasi, rantai proses mencurigakan dijalankan, atau URL *phishing* diakses, sensor menghitung nilai *hash* SHA-256 atau mengekstrak atribut proses/URL dan mengirimkan *event* ke Wazuh Manager.
2. **Korelasi Aturan di Wazuh Manager:** wazuh-analysisd mencocokkan *event* terhadap aturan FIM (*rule* 550, 554), aturan ekstensi ganda, aturan *canary ransomware* (*rule*

100202-100203), aturan rantai proses LOLBin (*rule* 100210-100213), maupun aturan akses *phishing*, lalu memicu skrip integrasi custom-n8n.

3. **Penyaringan Kebisingan dan Deduplikasi:** jembatan integrasi dan node awal n8n menyaring *path* aktivitas normal sistem (seperti /var/cache/ atau proses kontainer sementara) serta menerapkan jendela deduplikasi (*cooldown*) agar *alert* identik yang berulang dalam waktu singkat tidak membanjiri kuota API.
4. **Pemeriksaan Cache dan Koroborasi Threat Intelligence:** untuk alur *malware*, n8n terlebih dahulu memeriksa *cache* lokal ber-TTL diferensial dan indeks lokal misp-index. Apabila belum ter-cache, n8n memeriksa VirusTotal API v3 dan MalwareBazaar secara paralel, dengan *fallback* otomatis ke AlienVault OTX jika VirusTotal mengalami *rate-limit*. Untuk alur *phishing*, n8n memeriksa Google Safe Browsing API v4 dan URLScan.io.
5. **Klasifikasi Tingkat Keyakinan dan Otonomi (Tentukan Otonomi):** node tunggal Tentukan Otonomi mengevaluasi seluruh bukti *threat intelligence*, *rule level*, serta karakteristik berkas/proses untuk menetapkan *severity* dan tingkat otonomi tindakan sebagaimana dirumuskan pada Tabel 3.3.

Tabel 3.3 Matriks Tingkat Keyakinan dan Otonomi Respons

Severity dan Skor	Kondisi Pemicu dan Bukti Threat Intel	Tingkat Otonomi	Kebijakan Respons dan Tombol Telegram
CRITICAL (85-95)	VT >= 20 mesin, terdaftar di MalwareBazaar / MISP OSINT, YARA *match*, atau *Canary Ransomware*	auto-critical	Karantina otomatis seketika + Notifikasi Telegram + Tombol *Sudah ditinjau* (a:) & *Tandai FP / Restore* (f:)
HIGH - File (70-80)	VT 5-19 mesin atau OTX >= 3 *pulse* pada berkas baru/dimodifikasi	auto-high	Karantina otomatis + Notifikasi Telegram + Tombol *Sudah ditinjau* (a:) & *Tandai FP / Restore* (f:)
HIGH - Chain (75)	Deteksi rantai proses mencurigakan / LOLBin (*Rule* 100210-100213, Sysmon Event ID 1)	hitl-chain	Wajib persetujuan analis (HITL) ber-SLA 15 menit (mencegah penghentian paksa proses OS yang sah)
MEDIUM (40-55)	VT 1-4 mesin, OTX 1-2 *pulse*, API *degraded/rate-limited*, atau berkas eksekutabel dengan *hash* belum dikenal (*zero-day*)	hitl-review	Tahan eksekusi otomatis + Notifikasi Telegram + Tiga tombol: *Karantina file* (q:), *Sudah ditinjau* (a:), *Tandai FP* (f:) ber-SLA 15 menit
MEDIUM - Hosts (45)	Modifikasi berkas konfigurasi sistem /etc/hosts (termasuk aksi *sinkhole* domain)	manual-config	Dilarang mengarantina berkas sistem; hanya menyediakan tombol *Sudah ditinjau* (a:) & *Tandai FP* (f:)
LOW / BENIGN (5-25)	VT = 0 mesin (*Clean*) tanpa indikasi perilaku berbahaya, atau	observe	Dicatat pada *log* dan dasbor tanpa mengirim notifikasi interaktif ke

Severity dan Skor	Kondisi Pemicu dan Bukti Threat Intel	Tingkat Otonomi	Kebijakan Respons dan Tombol Telegram
	berkas teks non-eksekutabel biasa		Telegram (mereduksi *alert fatigue*)

6. **Penyusunan Analisis Kontekstual LLM Berbasis RAG Playbook:** node persiapan *prompt* mengambil cuplikan dokumen *playbook* penanganan insiden lokal sesuai tingkat keparahan dan jenis ancaman, lalu menginstruksikan LLM untuk menyusun ringkasan fakta indikator serta langkah remediasi dalam Bahasa Indonesia.
7. **Pendaftaran Registry SLA (alert-log):** setiap insiden dengan tingkat otonomi yang memerlukan atensi manusia didaftarkan ke layanan alert-log dengan batas waktu SLA 900 detik (15 menit).
8. **Pengiriman Notifikasi Interaktif:** n8n mengirimkan laporan insiden terformat ke Telegram Bot lengkap dengan susunan tombol *inline keyboard* yang sesuai dengan tingkat otonominya.

Adapun pemetaan cakupan lokasi pemantauan dan aturan deteksi sistem terhadap matriks taktik dan teknik **MITRE ATT&CK** disajikan pada Tabel 3.4.

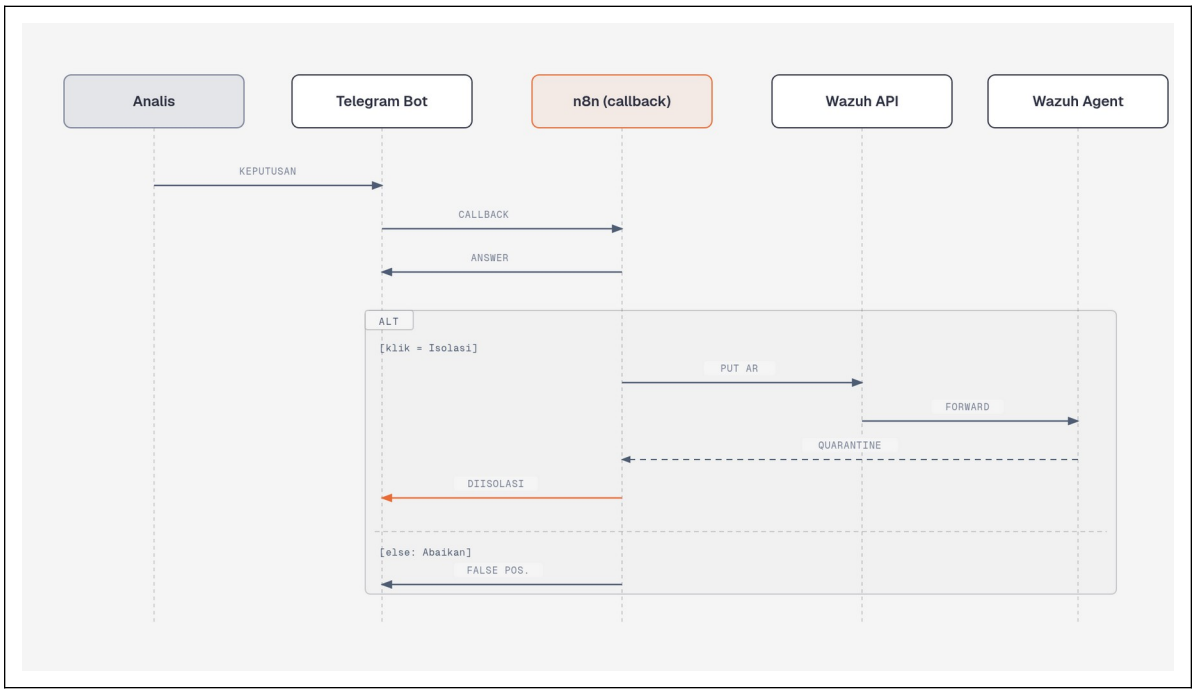
Tabel 3.4 Pemetaan Cakupan Deteksi terhadap MITRE ATT&CK

ID Teknik MITRE	Taktik dan Nama Teknik	Lokasi / Sumber Deteksi	Mekanisme Deteksi dan Respons
T1105 / T1204.002	*Command & Control / Execution: Ingress Tool Transfer & Malicious File*	~/Downloads, ~/Desktop, /tmp, Media USB (*Rule* 550, 554, YARA)	Pemantauan FIM *real-time* + reputasi SHA-256 (VT/MB/OTX/MISP) + karantina berkas (quarantine-file)
T1036.007	*Defense Evasion: Masquerading (Double File Extension)*	Direktori pengguna & unduhan (*Rule* 100205, sensor bit eksekusi)	Deteksi berkas berekstensi ganda (.pdf.exe, .docx.scr) dan biner tersembunyi + eskalasi triase
T1486	*Impact: Data Encrypted for Impact (Ransomware)*	Berkas umpan .soar-canary pada direktori pantauan (*Rule* 100202-100203)	Deteksi modifikasi/penghapusan berkas umpan *canary* secara seketika + respons kritis auto-critical
T1059 / T1218	*Execution / Defense Evasion: Scripting Interpreter & System Binary Proxy (LOLBin)*	Telemetri proses / Sysmon Event ID 1 (*Rule* 100210-100213)	Deteksi rantai proses cmd/powershell tersembunyi, certutil, mshta, regsvr32, rundll32 + eskalasi hitl-chain
T1053.003 / T1543.002	*Persistence: Scheduled Task (Cron) & Systemd Service*	/etc/cron.*, /var/spool/cron, /etc/systemd/system, ~/.ssh/	Pemantauan FIM pada direktori persistensi layanan dan kunci autentikasi SSH
T1566.002 / T1189	*Initial Access: Spearphishing Link &*	*Log* akses URL, berkas .url/.lnk/.desktop, &	Verifikasi reputasi URL terhadap Google Safe Browsing & URLScan.io + pemblokiran domain

ID Teknik MITRE	Taktik dan Nama Teknik	Lokasi / Sumber Deteksi	Mekanisme Deteksi dan Respons
	Drive-by Compromise	umpan URLhaus	(block-domain)
T1565.001	*Impact: Stored Data Manipulation (Hosts Configuration)*	Berkas resolusi nama /etc/hosts (Linux) & hosts (Windows)	Pemantauan integritas berkas hosts dengan tingkat otonomi manual-config dan pengenalan *sinkhole* SOAR

3.8 Desain Active Response Human-in-the-Loop dan Integrasi Telegram Bot

Integrasi Telegram Bot dirancang sebagai antarmuka kendali dua arah (*bidirectional command interface*) antara sistem SOAR dan analis keamanan. Alur interaksi berurutan saat insiden muncul hingga dieksekusinya tindakan pada *endpoint* ditunjukkan pada *sequence diagram* Gambar 3.5.



Gambar 3.5 Sequence Active Response Human-in-the-Loop

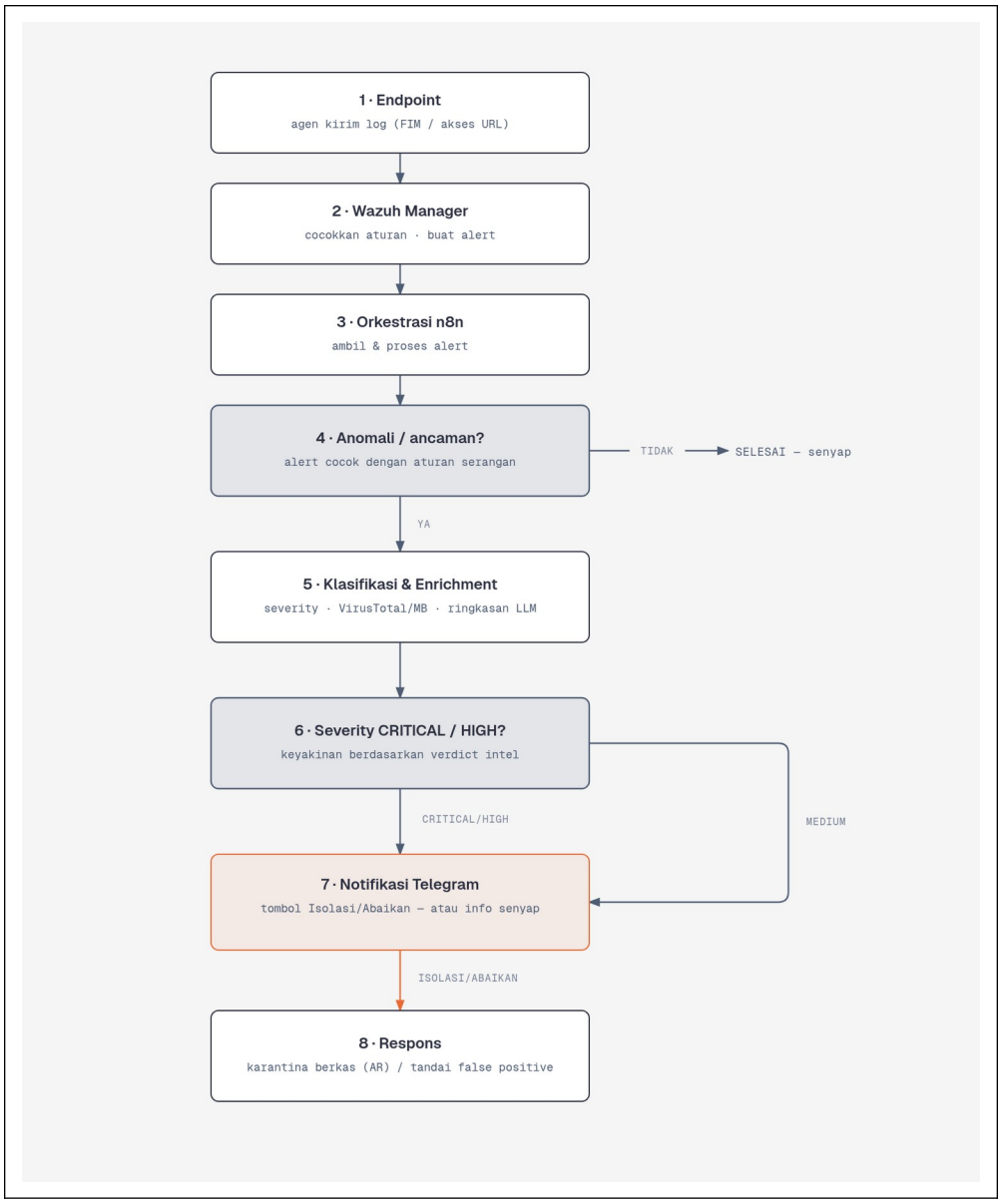
Sebagaimana diilustrasikan pada Gambar 3.5, tahapan teknis mekanisme *Active Response human-in-the-loop* dirancang sebagai berikut.

1. **Pengiriman Notifikasi Berbasis Tingkat Otonomi:** untuk insiden yang memerlukan keputusan manusia (hitl-review, hitl-chain, atau manual-config) maupun umpan balik pasca-karantina otomatis (auto-critical dan auto-high), n8n mengirim pesan Telegram dilengkapi *inline keyboard* dengan skema callback_data ringkas di bawah batas 64 byte, yaitu q:<agentId> (Karantina File), a:<agentId> (Sudah Ditinjau /

Acknowledge), f:<agentId> (Tandai False Positive / Restore), serta pblk:<agentId> dan pign:<agentId> untuk pemblokiran atau pengabaian domain *phishing*.

2. **Penjemputan Klik Tombol Keluar (tg-callback-poller):** karena server beroperasi di balik NAT/Tailscale tanpa mengekspos *webhook* n8n ke internet publik, layanan tg-callback-poller menjalankan *long-polling* keluar (getUpdates) ke Telegram API dan meneruskan setiap kejadian callback_query ke *webhook* lokal /webhook/tg-callback pada n8n.
3. **Validasi Keamanan dan Penanganan Keputusan:** *Workflow Telegram Callback Handler* memverifikasi otorisasi pengirim, mengekstrak kode aksi, identitas *agent*, serta lokasi berkas atau domain dari pesan, lalu memanggil answerCallbackQuery untuk memberikan respons instan pada layar Telegram analis.
4. **Eksekusi Isolasi, Blokir Domain, atau Pemulihan Berkas (*Restore*):** apabila analis menekan tombol karantina (q:), n8n melakukan autentikasi JWT ke Wazuh API (POST /security/user/authenticate) dan memanggil PUT /active-response dengan perintah !quarantine-file. Daemon wazuh-execd pada *endpoint* memindahkan berkas ke /var/ossec/quarantine/, mengubah izin akses menjadi chmod 000, serta mencatat lokasi asli dan *hash* SHA-256 pada berkas metadata .meta. Apabila analis menekan tombol f: pada berkas yang telanjur dikarantina otomatis, sistem dapat memicu prosedur *restore* yang memverifikasi keutuhan *hash* SHA-256 sebelum mengembalikan berkas ke lokasi semula. Untuk ancaman *phishing*, perintah !block-domain menyisipkan aturan *sinkhole* 0.0.0.0 <domain> ke dalam berkas /etc/hosts.
5. **Pembaruan Status Audit dan Eskalasi SLA:** setiap klik tombol langsung memperbarui catatan keputusan pada layanan alert-log (mencatat nama analis, tindakan yang dipilih, dan selisih waktu respons) serta memperbarui teks pesan di Telegram. Apabila tiket berstatus pending melewati batas waktu 15 menit tanpa keputusan, *Workflow Eskalasi SLA* mengirimkan peringatan pengingat otomatis ke kanal Telegram.

Rancangan antarmuka pesan interaktif Telegram Bot beserta alur penanganannya ditunjukkan pada Gambar 3.6.



Gambar 3.6 Desain Integrasi Telegram Bot

Dengan desain pada Gambar 3.6, setiap insiden meninggalkan jejak audit lengkap di tujuh titik pencatatan, yaitu *log* agen (ossec.log / soar-agent), *log alert* Manager (alerts.log), *log Active Response* (active-responses.log), *log* jembatan integrasi (integrations.log), riwayat eksekusi *workflow* n8n, basis data keputusan alert-log, serta riwayat pesan Telegram.

3.9 Perancangan Modul Riwayat dan Monitoring Event

Selain saluran notifikasi *real-time* di Telegram, sistem menyediakan modul **riwayat event dan pemantauan armada** pada *dashboard* web (fleet-monitor) serta antarmuka terminal (fleet-tui) agar administrator dapat meninjau seluruh aktivitas keamanan pada rentang

waktu tertentu (24 jam, 7 hari, 30 hari, 90 hari, maupun rentang tanggal kustom). Modul ini dirancang dengan mengintegrasikan tiga kapabilitas utama sebagai berikut.

1. **Penyimpanan Hibrida dan Deduplikasi Lintas-Sumber:** menggabungkan data *event* yang telah diperkaya oleh *pipeline* SOAR pada basis data **SQLite** lokal (dengan masa retensi bawaan 90 hari dan pemangkasan otomatis) bersama arsip *alert* historis dari **Wazuh Indexer** (indeks `wazuh-alerts-*`). Data dari kedua sumber dideduplikasi berdasarkan kombinasi `agent_id`, *hash* SHA-256, dan lokasi berkas sehingga insiden yang sama tidak ditampilkan ganda.
2. **REST API Riwayat dan Kendali Karantina:** menyajikan data melalui *endpoint* GET `/api/events/history` dengan dukungan paginasi sisi server, penyaringan per *severity* atau tingkat otonomi, agregasi statistik harian untuk visualisasi grafik, serta *endpoint* manajemen karantina yang memungkinkan analis memicu pemulihan berkas (*restore*) langsung dari antarmuka dasbor. Apabila Wazuh Indexer sedang tidak terjangkau, layanan tetap berfungsi secara normal menggunakan sumber data SQLite lokal (*graceful degradation*).
3. **Pemantauan Kesehatan Armada Lintas-Platform:** memantau detak kesehatan (*heartbeat*), penggunaan CPU dan memori, serta status keterhubungan seluruh *endpoint* Linux dan Windows yang menjalankan Wazuh Agent maupun `soar-agent` dalam satu tampilan terpusat.

DAFTAR PUSTAKA

Alves, I., dkk. (2026). Designing a Distributed Wazuh-Based Security Incident Detection Architecture. *Journal of Cybersecurity and Information Systems*, 14(1), 45-58.

Antara, G. P., & Rachmawati, I. D. A. (2024). Implementasi dan Analisis Wazuh Sebagai Intrusion Detection System (IDS) dan Platform Monitoring. *Jurnal Informasi, Sains dan Teknologi*, 7(2), 290-303. https://doi.org/10.55606/isaintek.v7i2.301

Ariyanto, Y., Syaifudin, Y. W., Ratsanjani, M. H., Muladawila, A. R., Fatmawati, T., Saputra, P. Y., & Setiadi, C. (2025). Cyber Threat Detection and Automated Response Using Wazuh and Telegram API. *Matrik: Jurnal Manajemen, Teknik Informatika dan Rekayasa Komputer*, 25(1), 173-188. https://doi.org/10.30812/matrik.v25i1.5610

Artika, D. A., Rumahorbo, D. R., Haikal, M. H. A.-M., & Kiswanto, D. (2025). Implementasi Sistem Keamanan Website dengan Analisis Log dan Deteksi Aktivitas Anomali Menggunakan Isolation Forest. *Jurnal Informatika dan Teknik Elektro Terapan (JITET)*, 13(3S1). https://doi.org/10.23960/jitet.v13i3S1.8133

Chhetri, M. B., Tariq, S., & Nepal, S. (2024). *Human-AI Teaming in Security Operations Centers: A Survey of Challenges and Opportunities*. ACM Computing Surveys, 56(9), 1-36.

Cisco Talos. (2026). *Cisco Talos Incident Response & Threat Intelligence Report Q1-Q3 2026: Abuse of Workflow Automation Platforms and Emerging Malware Trends*. Cisco Systems, Inc.

Farrel, F. I. F., Mardianto, I., & Qamar, A. S. (2024). Implementation of Security Information & Event Management (SIEM) Wazuh with Active Response and Telegram Notification for Mitigating Brute Force Attacks on The GT-I2TI USAKTI Information System. *Intelmatics*, 4(1), 1-7. https://doi.org/10.25105/itm.v4i1.18529

Gartner. (2023). *Market Guide for Security Orchestration, Automation and Response (SOAR) Solutions*. Gartner Research.

Habibzadeh, M., dkk. (2026). Large Language Models in Cyber Defense and Security Operations Center Automation: A Comprehensive Survey. *Computers & Security*, 148, 104118.

Hammar, K., Stadler, R., & Alpcan, T. (2025). Retrieval-Augmented Generation and Constrained Planning for Incident Response Using Large Language Models. *IEEE Transactions on Network and Service Management*, 22(2), 1410-1425.

Heluka, H. D., & Sulistyo, W. (2023). Perancangan dan Implementasi Security Information and Event Management (SIEM) pada Layanan Virtual Server. *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIIK)*, 10(4), 781-790.

MITRE. (2026). *MITRE ATT&CK Enterprise Matrix v15.1*. The MITRE Corporation. https://attack.mitre.org/

Mohsin, A., Janicke, H., Nepal, S., & Tariq, S. (2025). Trusted Autonomy in Human-AI Collaborative Security Operations Centers: Mapping Confidence to Response Autonomy. *Computers & Security*, 142, 103892.

n8n GmbH. (2026). *n8n Workflow Automation Documentation (v2.42)*. https://docs.n8n.io/

Nadig, R., & Vignesh, S. (2026). Enhancing SIEM Capabilities through Open-Source SOAR Integration for Automated Incident Response. *International Journal of Information Security and Privacy*, 20(1), 112-128.

NIST. (2024). *The NIST Cybersecurity Framework (CSF) 2.0 & Computer Security Incident Handling Guide (SP 800-61 Rev. 3)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.CSWP.29

OWASP. (2026). *OWASP Top 10 for Large Language Model Applications & Agentic AI Security Guidelines (v2.0)*. Open Worldwide Application Security Project. https://genai.owasp.org/

Prasetyo, O. D., Trisnawan, P. H., & Bhawiyuga, A. (2023). Uji Kinerja Host-Based Intrusion Detection System Wazuh terhadap Serangan Brute Force dan DoS. *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer (J-PTIIK)*, 7(6), 2810-2819.

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Puteri, N. A., dkk. (2024). Konsep dan Penerapan Keamanan Siber Berbasis NIST Cybersecurity Framework pada Organisasi Digital. *Jurnal Keamanan Siber dan Kriptografi*, 6(1), 33-44.

Saputra, F. A., dkk. (2024). Penerapan Wazuh sebagai SIEM Terintegrasi Telegram Bot untuk Deteksi dan Visualisasi Log Keamanan Real-time (Studi Kasus PeTIK Jombang). *Jurnal Sains dan Teknologi Informasi*, 9(2), 101-110.

Singh, P., dkk. (2025). Empirical Evaluation of Large Language Models as Cognitive Assistants for SOC Analysts: An Analysis of 3,090 Triage Queries. *Proceedings of the IEEE Symposium on Security and Privacy (SP)*, 1890-1907.

Srinivas, A., dkk. (2025). Benchmarking LLM Explainability and Hallucination Risks in Automated Alert Triage. *ACM Workshop on Artificial Intelligence and Security (AISec)*, 74-85.

Tariq, S., Chhetri, M. B., Nepal, S., & Paris, C. (2025). Mitigating Alert Fatigue in Security Operations Centers: A Survey of Intelligent Triage and Enrichment Approaches. *IEEE Communications Surveys & Tutorials*, 27(1), 412-445.

Telegram. (2026). *Telegram Bot API Reference*. Telegram Messenger Inc. https://core.telegram.org/bots/api

Thakker, K., Patel, D., & Shah, M. (2025). Automated Defense Against Application-Layer Attacks on Windows Endpoints Using Wazuh SIEM and Shuffle SOAR. *Journal of Information Security and Applications*, 82, 103741.

VirusTotal. (2026). *VirusTotal API v3 Reference Documentation*. Chronicle Security / Google Cloud. https://docs.virustotal.com/reference/overview

Wazuh Inc. (2026). *Wazuh Documentation v4.14: File Integrity Monitoring, Threat Detection, and Active Response*. https://documentation.wazuh.com/current/